

Manual Ejecutivo de Construcción Android: Fénix Games

1. Decisión técnica final

El presente documento constituye el manual arquitectónico y de ejecución técnica definitivo para la construcción del proyecto "Fénix Games". La directiva innegociable es el costo operativo cero mensual. Se descarta cualquier infraestructura basada en la nube, incluyendo Firebase, Supabase, Vercel, Render o APIs externas con niveles de uso gratuitos. La aplicación no requerirá infraestructuras externas para la autenticación ni para la transferencia de estado de la aplicación, garantizando resiliencia absoluta frente a la obsolescencia programada o la caída de servidores de terceros.

El stack tecnológico seleccionado y cerrado para el repositorio de código abierto fenix-games se fundamenta en un ecosistema nativo puro. La plataforma de desarrollo es Android Nativo, utilizando Kotlin como lenguaje principal. La capa de presentación se construirá exclusivamente con Jetpack Compose, adoptando Clean Architecture junto con un patrón Modelo-Vista-Intención (MVI) unidireccional. Esta decisión garantiza una separación limpia y testeable de los estados de interfaz de usuario, optimizando las recomposiciones mediante herramientas nativas.

Para manejar la concurrencia y la propagación de estados, se emplearán Kotlin Coroutines y StateFlow, asegurando un rendimiento fluido a 60 cuadros por segundo. La persistencia de datos local, pilar fundamental de la operatividad offline, recaerá sobre Room Database operando sobre SQLite. Esto actuará como el mecanismo central inmutable para los datos de sesión, contenido y puntuación, mientras que DataStore gestionará los metadatos de preferencias y configuraciones menores.

La estructuración e ingesta de datos prefabricados se basará en archivos JSON locales, procesados asincrónicamente mediante la biblioteca kotlinx.serialization, aprovechando sus capacidades polimórficas para manejar distintas clases de cartas de juego. Para permitir el crecimiento del contenido sin depender de servidores, se integrará el Storage Access Framework (SAF) nativo de Android. Esto habilitará la importación y exportación de paquetes de contenido (.party o .json) y la extracción de bitácoras de diagnóstico directamente desde y hacia el sistema de archivos del usuario.

Las fases avanzadas del proyecto introducirán una topología de red local descentralizada (Local-First Architecture). Para ello, se levantará un servidor HTTP y WebSocket local embebido utilizando el framework Ktor. Este servidor correrá sobre un Servicio en Primer Plano (Foreground Service) configurado con los tipos requeridos por Android 14 para evitar su finalización por parte del sistema operativo. El descubrimiento de nodos en la red local omitirá mDNS frágiles en favor de la generación estática de códigos QR mediante zxing-core, los cuales serán interpretados por los dispositivos clientes utilizando el escáner neuronal fuera de línea de ML Kit. Finalmente, la infraestructura de versionado residirá en GitHub, empleando GitHub Actions como mecanismo automatizado y sin costo para el ensamblado continuo del artefacto APK.

2. Preparación del repositorio GitHub

La infraestructura de versionado y control de código fuente debe establecerse meticulosamente antes de escribir la primera línea de código en Kotlin. El repositorio oficial se denominará `fenix-games`, con el nombre visible del proyecto fijado como "Fénix Games". La creación del repositorio en GitHub debe inicializarse como un entorno vacío para permitir una configuración manual de la estructura base.

Para inicializar el entorno local, se ejecutarán los siguientes comandos Git concretos en la terminal del sistema operativo de desarrollo. Primero, se crea el directorio y se inicializa el repositorio con la rama principal ajustada a los estándares modernos: `mkdir fenix-games`, seguido de `cd fenix-games`, y luego `git init` y `git branch -m main`.

Inmediatamente, se debe configurar el archivo `.gitignore` específico para entornos Android. Este archivo debe ser exhaustivo para evitar la subida de binarios compilados, directorios IDE locales y, crucialmente, archivos de firma de seguridad que contengan contraseñas. El archivo debe excluir directorios como `/build`, `/.idea/`, `*.iml`, `local.properties`, `*.apk`, `*.aab`, y cualquier archivo de certificado como `*.jks` o `*.keystore`. Esta es la principal medida de protección básica contra la filtración de secretos, garantizando que el repositorio se mantenga limpio y seguro. El repositorio debe contener archivos de documentación de gobernanza claros. Se creará un `README.md` inicial que detalle las instrucciones de compilación arquitectónica, declarando explícitamente la naturaleza offline y de costo cero del proyecto. Asimismo, se incorporará un archivo `CHANGELOG.md` basado en el estándar "Keep a Changelog" para registrar las evoluciones por fases, y un archivo `LICENSE` (por ejemplo, MIT o GPLv3) para dictaminar los términos de uso y distribución del código.

La estrategia de ramas (Branching Strategy) seguirá un modelo adaptado a entregas guiadas por fases. Existirá una rama `main` que contendrá exclusivamente código de producción validado y estable (generador del APK final). Una rama `develop` actuará como el canal de integración principal. Para el desarrollo diario, se exigirá la creación de ramas efímeras siguiendo la convención `feature/fase-X-nombre-microfase` (por ejemplo, `feature/fase-1-modelos-dominio` o `fix/fase-2-exportacion-saf`).

La estrategia de commits debe regirse por la convención "Conventional Commits" para facilitar la lectura del historial. Se utilizarán prefijos obligatorios como `feat`: para nuevas características, `fix`: para correcciones de errores, `chore`: para tareas de mantenimiento o actualización de dependencias, y `docs`: para documentación. Un formato de commit válido sería:

`feat(caja-negra): implementar captura global de crashes offline`. Para validar cada push, los desarrolladores deberán ejecutar las pruebas de compilación locales mediante el comando `./gradlew assembleDebug` garantizando que el árbol de trabajo esté limpio.

La estrategia de tags y releases se sincronizará con el final de cada fase principal delineada en este manual. Al completar la Fase 1, se ejecutará el comando `git tag -a v1.0.0-fase1 -m "Versión validada de la Fase 1: Núcleo Offline"`. Estos tags coincidirán con la publicación de Releases en GitHub, donde se adjuntará el artefacto APK compilado para pruebas.

Finalmente, se configurará el uso opcional pero altamente recomendado de GitHub Actions para lograr integración continua con costo cero absoluto. Se creará un archivo de flujo de trabajo en la ruta `.github/workflows/android-build.yml`. Este script aprovisionará un entorno virtual Linux gratuito provisto por GitHub, configurará el JDK necesario (por ejemplo, Java 17), ejecutará el comando de ensamblado de Gradle (`./gradlew assembleDebug`), y subirá el archivo `app-debug.apk` resultante como un artefacto descargable al finalizar el flujo. Esto permite a los testers descargar el ejecutable directamente desde el repositorio sin depender de

infraestructuras de compilación externas.

3. Fase 0: Inicialización del proyecto Android

Esta fase fundacional establece el andamiaje del proyecto Android, asegurando que las dependencias, herramientas y estructuras arquitectónicas estén correctamente inyectadas antes de comenzar la codificación lógica.

Para la microfase 0.1, enfocada en crear el proyecto Android, el objetivo es inicializar la estructura nativa utilizando Android Studio. Se deben crear los archivos base modificando la configuración predeterminada para asegurar un proyecto completamente basado en Kotlin DSL. El criterio de aceptación exige que el proyecto compile sin errores tras la inicialización inicial. El error más común en este punto es la configuración incorrecta del SDK objetivo; se corrige asegurando que el `compileSdk` y el `targetSdk` apunten a la API 34 o superior, preparando el terreno para las restricciones de servicios en primer plano de Android 14. La prueba mínima es la ejecución de una sincronización de Gradle exitosa.

La microfase 0.2 aborda la configuración de Gradle. El objetivo es centralizar la gestión de versiones utilizando el archivo `gradle/libs.versions.toml` (Version Catalogs). Se modificarán los archivos `build.gradle.kts` a nivel de proyecto y módulo. El criterio de aceptación es que todas las dependencias se referencien mediante literales de catálogo (ej. `libs.androidx.core.ktx`) eliminando cadenas "mágicas" de versiones. Un error común es la desalineación de plugins; esto se corrige asegurando que las declaraciones de alias en el catálogo coincidan con los IDs de los plugins. La prueba mínima es ejecutar `./gradlew tasks` en la terminal.

En la microfase 0.3, se configura Kotlin. El objetivo es habilitar las características avanzadas del lenguaje y la concurrencia. En el archivo `build.gradle.kts` del módulo `app`, se agregarán las dependencias para `kotlinx-coroutines-android` y `kotlinx-coroutines-core`. El criterio de aceptación dicta que el proyecto debe permitir la invocación de `CoroutineScope` y el manejo de `Dispatchers` sin errores de resolución. La prueba manual implica crear una función de prueba suspendida trivial para validar la compilación cruzada.

La microfase 0.4 se centra en configurar Jetpack Compose. El objetivo es habilitar el framework declarativo de UI. Se deben modificar las directivas de compilación en el archivo Gradle añadiendo el bloque `buildFeatures { compose = true }` e incorporando las bibliotecas del compilador de Compose. El criterio de aceptación exige la capacidad de utilizar la anotación `@Composable`. Un error habitual es un conflicto de versiones entre el compilador de Kotlin y el compilador de Compose; se corrige consultando la matriz de compatibilidad oficial de Google.

La microfase 0.5 define la configuración de Material 3. El objetivo es establecer un sistema de diseño robusto y adaptable. Se modificarán los archivos dentro del paquete `ui/theme/`, incluyendo esquemas de color, tipografía y formas. El criterio de aceptación requiere que se aplique el componente `MaterialTheme` envolviendo toda la jerarquía de la interfaz. La prueba mínima es visualizar un cambio de color primario en el texto de prueba renderizado en la pantalla.

La microfase 0.6 establece Navigation Compose. El objetivo es preparar el enrutamiento de la aplicación sin depender de la fragmentación de Android. Se añadirá la dependencia de navegación y se creará el archivo `AppNavigation.kt` configurando el `NavHost` base. El criterio de aceptación es que se pueda navegar programáticamente entre dos composables vacíos. El error común es perder el contexto de navegación al recomponer; se soluciona asegurando que el `NavController` sea recordado en el nivel superior de la jerarquía.

En la microfase 0.7, se configura Room. El objetivo es integrar el ORM local y offline. Se

añadirán las dependencias de room-runtime, room-ktx, y el procesador de anotaciones room-compiler a través de KSP (Kotlin Symbol Processing). El archivo a crear es un esbozo inicial de AppDatabase.kt. El criterio de aceptación es que la base de datos pueda ser instanciada como un singleton. Un error frecuente es omitir el plugin KSP; se corrige añadiendo id("com.google.devtools.ksp") en el bloque de plugins de Gradle.

Para la microfase 0.8, se configura DataStore. El objetivo es reemplazar SharedPreferences para el almacenamiento de preferencias de usuario. Se añadirá la dependencia de datastore-preferences y se creará una abstracción SettingsManager.kt. El criterio de aceptación requiere la lectura asíncrona de un valor booleano (ej. "primera apertura"). La prueba mínima es inyectar un valor por defecto y leerlo en el inicio de la app.

La microfase 0.9 introduce kotlinx.serialization. El objetivo es habilitar el análisis y generación de JSON ultrarrápido y sin reflexión. Se añadirá el plugin de serialización de Kotlin y la dependencia JSON. Se crearán clases de datos de prueba anotadas con @Serializable. El criterio de aceptación es que un string JSON simple pueda convertirse en una instancia de objeto Kotlin. El error común es olvidar la anotación en estructuras anidadas, solucionable revisando el árbol de herencia.

En la microfase 0.10, se diseña la estructura base de paquetes. El objetivo es aplicar Clean Architecture. Se crearán los directorios internos de Kotlin: domain, data, ui, core, y games. El criterio de aceptación es que esta estructura esté reflejada en el control de versiones sin contener clases de negocio reales aún. La prueba es visual, asegurando un andamiaje lógico que previene el código espagueti.

La microfase 0.11 aborda la creación de la pantalla inicial mínima. El objetivo es tener un punto de entrada visual funcional. Se creará HomeScreen.kt conteniendo un mensaje de bienvenida ("Fénix Games") y un botón inactivo, soportado por un Scaffold. El criterio de aceptación es que la interfaz se muestre limpia y adherida al tema de Material 3 previamente configurado.

Para la microfase 0.12, se ejecuta el primer build local. El objetivo es asegurar que toda la infraestructura de la Fase 0 consolida un ejecutable nativo. El comando es ./gradlew assembleDebug. El criterio de aceptación es la generación de un archivo APK en el directorio de salidas (outputs) con un peso razonable (inferior a 15 MB) y un registro de compilación libre de advertencias críticas sobre dependencias obsoletas.

La microfase 0.13 formaliza el primer commit. El objetivo es sellar la fundación del proyecto en el repositorio GitHub. Se ejecutarán los comandos git add. y git commit -m "chore(init): inicialización del andamiaje Android, Compose, Room y Ktor dependencias". El criterio de aceptación es que la rama main del repositorio remoto refleje el proyecto estructurado.

En la microfase 0.14, se procede a la validación en emulador. El objetivo es constatar el comportamiento en un entorno controlado. La prueba mínima exige lanzar el APK generado en un Dispositivo Virtual Android (AVD) de API reciente. El criterio de aceptación determina que la aplicación debe abrir instantáneamente mostrando la HomeScreen sin presentar cierres forzados por falta de configuración del tema.

Finalmente, la microfase 0.15 requiere la validación en móvil real. El objetivo es probar la fricción cero en silicio real. El archivo APK se transferirá a un teléfono físico mediante ADB. El criterio de aceptación demanda que la instalación sea exitosa y que la aplicación se ejecute sin solicitar permisos excesivos. La prueba manual incluye abrir la aplicación y minimizarla varias veces para asegurar que el ciclo de vida base opera correctamente, marcando el éxito total de la Fase 0.

4. Fase 1: Núcleo offline mínimo

La Fase 1 establece las tuberías lógicas y de datos que permiten a Fénix Games operar sin ninguna conexión a internet, basándose en flujos asíncronos unidireccionales de arquitectura MVI.

En la microfase 1.1, se estructuran los Modelos de dominio. El objetivo es aislar la lógica conceptual del proyecto. Se crearán archivos de clases inmutables que no posean anotaciones de Room ni de Jetpack Compose. Esto asegura que la base algorítmica de los juegos respete la Clean Architecture. El criterio de aceptación estipula que cualquier modificación en la UI no requerirá alterar estas entidades fundamentales.

La microfase 1.2 define el GameMode. El objetivo es crear una taxonomía estricta de las modalidades lúdicas. Se implementará una clase sellada (sealed class) en Kotlin. El criterio de aceptación exige que esta clase contenga objetos o subclases que representen cada subjuego (por ejemplo, TruthOrDare, NeverHaveEver), permitiendo un manejo exhaustivo mediante bloques when en los enrutadores lógicos sin sentencias else inseguras.

Para la microfase 1.3, se implementa el modelo Player. El objetivo es representar a los participantes humanos. El archivo Player.kt incluirá propiedades como un UUID inmutable, un nombre, un avatar referencial y, crucialmente, su ConsentState y puntuación actual. El criterio de aceptación es que el identificador evite colisiones cuando se transfiera en el futuro a la red local.

La microfase 1.4 establece la GameSession. El objetivo es aglutinar el estado global efímero de una partida en curso. Este modelo contendrá la lista de jugadores activos, el índice de turno y el modo de juego en ejecución. Su criterio de aceptación dicta que debe ser clonable mediante la función copy() de Kotlin para facilitar la emisión de nuevos estados en la arquitectura MVI sin mutar las referencias originales.

En la microfase 1.5, se consolida la entidad Round. El objetivo es fragmentar el flujo del tiempo de juego. Definirá al jugador activo, al jugador objetivo (si aplica) y la fase actual de interacción. El criterio de aceptación es que represente atómicamente el estado en el que la UI debe reaccionar.

La microfase 1.6 incorpora el objeto Score. El objetivo es gestionar las sumatorias matemáticas. Se crearán archivos de manipulación de puntos que se integren directamente con la actualización de la lista de jugadores de la sesión. El criterio de aceptación exige evitar operaciones aritméticas directamente en los composables de la vista.

En la microfase 1.7, se desarrolla el TimerEngine. El objetivo es proporcionar cronometría resiliente offline. Se programará una Coroutine que opere dentro del viewModelScope, emitiendo valores de tiempo restante a un StateFlow<Long>. El criterio de aceptación primordial es que el temporizador no debe reiniciar su conteo interno al ocurrir cambios de configuración del dispositivo, como la rotación de pantalla.

La microfase 1.8 implementa el Randomizer. El objetivo es administrar la entropía sin dependencia de librerías externas predictibles. Se implementará un algoritmo robusto, como Fisher-Yates, encapsulado en una clase utilitaria. El criterio de aceptación requiere que devuelva permutaciones uniformes para la distribución justa de roles de juego y cartas.

La microfase 1.9 diseña el ContentPackManager. El objetivo es intermediar entre los JSON estáticos y la base de datos SQL. Este gestor asíncrono se encargará de delegar tareas transaccionales pesadas al Hilo IO de Android. Su criterio de aceptación dicta que debe ser el único punto de contacto cuando la aplicación solicita ensamblar un mazo de cartas para la sesión.

En la microfase 1.10, se desarrolla la carga JSON desde assets. El objetivo es absorber la base masiva de contenido lúdico en el primer arranque. El archivo utilizará el contexto para abrir un InputStream desde el directorio /assets y lo convertirá en objetos de dominio utilizando `Json.decodeFromString` de `kotlinx.serialization`. El criterio de aceptación asegura que un JSON de miles de líneas se analice sin bloquear el renderizado visual.

La microfase 1.11 automatiza la prepoblación de Room (Room prepopulation). El objetivo es volcar los objetos analizados del JSON hacia SQLite para permitir consultas submilisegundo. Se empleará un `RoomDatabase.Callback` para ejecutar comandos masivos `insertAll()` tras la creación inicial de la base de datos. El criterio de aceptación exige que la aplicación presente un indicador visual de carga durante este proceso único, bloqueando el acceso al inicio hasta que los datos estén completamente asentados.

Para la microfase 1.12, se erige el registro de cartas usadas (UsedCard registry). El objetivo es impedir el agotamiento cíclico y molesto de cartas repetidas. Se implementará manteniendo en memoria una colección `Set<UUID>` de las cartas consumidas en la sesión, la cual será anexada como parámetro de exclusión en las consultas DAO (`WHERE id NOT IN (:usedIds)`). El criterio de aceptación prueba que es estadísticamente imposible obtener la misma carta dos veces hasta que el mazo se agote matemáticamente.

La microfase 1.13 consolida el `SessionManager`. El objetivo es crear la máquina de estados maestro del paradigma MVI. Este manejador procesará intenciones (Intents como `NextTurn`, `AddScore`) y reducirá el estado global exponiéndolo como un único flujo inmutable a la UI. El criterio de aceptación requiere que todo cambio en la pantalla sea estrictamente reactivo a los despachos de este administrador.

La microfase 1.14 crea la navegación mínima. El objetivo es estructurar el flujo de pantallas fundacionales: desde la Home, pasando por el formulario de creación de jugadores, la selección de modo de juego, hasta aterrizar en la vista lúdica vacía. El criterio de aceptación es que la pila de retroceso (backstack) funcione lógicamente al presionar el botón atrás nativo de Android.

Finalmente, la microfase 1.15 ejecuta la validación en móvil real. El objetivo es probar la latencia cero. El dispositivo anfitrión operará en Modo Avión. El tester añadirá seis jugadores, navegará por los menús y cerrará la aplicación abruptamente para confirmar que el arranque del ORM Room fue definitivo. La prueba debe evidenciar cero paradas (crashes) por falta de resolución DNS o peticiones REST caídas, confirmando que la aplicación es nativamente autárquica.

5. Fase 2: Caja Negra Fénix

La Caja Negra Fénix es la herramienta de diagnóstico fundamental que permite investigar anomalías sistémicas en entornos sin internet, evitando la dependencia de herramientas como Firebase Crashlytics.

En la microfase 2.1, se diseña el `BlackBoxLogger`. El objetivo es establecer un embudo único centralizado para todo registro de la aplicación. Se creará una interfaz genérica de logging y su implementación concreta para escritura en disco. El criterio de aceptación exige que todas las clases de la aplicación utilicen este logger abstracto en lugar de las llamadas nativas de `android.util.Log`.

La microfase 2.2 aborda el modelado del `DiagnosticEvent`. El objetivo es clasificar forensemente las fallas. Se implementará una clase sellada con tipos como `NavigationError`, `DatabaseFault`, o `NetworkAnomaly`. El criterio de aceptación determina que cada evento posea

una marca temporal (timestamp) precisa generada localmente.

Para la microfase 2.3, se establece el LogLevel. El objetivo es priorizar la severidad. Se programará una estructura de enumeración abarcando niveles como INFO, WARNING, ERROR, y FATAL. El criterio de aceptación dicta que el sistema de escritura pueda ser configurado dinámicamente para registrar solo umbrales específicos dependiendo del entorno de compilación.

La microfase 2.4 se encarga de crear el almacenamiento local de logs. El objetivo es persistir físicamente los rastros. Se codificarán funciones de entrada/salida (I/O) respaldadas por Dispatchers secundarios para escribir apéndices de texto secuenciales en el directorio de almacenamiento interno privado de la aplicación (Context.filesDir). El criterio de aceptación es que la escritura en disco jamás bloquee ni retrase las animaciones de la interfaz Jetpack Compose.

La microfase 2.5 captura errores no fatales. El objetivo es anotar desviaciones lógicas que no detienen el proceso. Se envolverán las rutinas sensibles (como el análisis JSON o la inserción de datos) en bloques runCatching {}, y en caso de fallo, se despachará el evento silenciosamente hacia el BlackBoxLogger. El criterio de aceptación exige que el usuario reciba un estado neutral reactivo sin alertar diálogos hostiles del sistema.

La microfase 2.6 es vital: capturar crashes globales. El objetivo es registrar el rastro de la pila (Stack Trace) antes de la muerte del proceso operativo de Android. Se modificará la clase Application instanciando y configurando Thread.setDefaultUncaughtExceptionHandler. Esta rutina interceptará la caída, enviará la cadena de error masiva al disco local asincrónicamente y finalmente invocará el manejador original o cerrará el proceso mediante exitProcess para evitar interfaces congeladas.

La microfase 2.7 registra eventos de navegación. El objetivo es crear un mapa del historial del usuario (breadcrumbs). Se integrará un interceptor en Navigation Compose que alimentará la Caja Negra con cada transición de pantalla.

La microfase 2.8 se orienta a registrar eventos de juego. El objetivo es auditar anomalías en la entropía o la lógica. El SessionManager emitirá entradas silentes (INFO) cuando asigne roles o vacíe mazos.

La microfase 2.9 documenta errores de Room. El objetivo es identificar migraciones rotas o violaciones de claves foráneas. Se implementarán bloques try-catch alrededor de las transacciones DAO conflictivas.

La microfase 2.10 registra errores JSON. El objetivo es identificar paquetes creados por la comunidad que rompen el esquema polimórfico preestablecido. Cualquier SerializationException interceptada será anotada en el log con el fragmento problemático para diagnóstico.

La microfase 2.11 crea la DiagnosticScreen. El objetivo es proporcionar una interfaz rústica para visualizar la traza en tiempo real. Se codificará una pantalla Compose en formato lista que consume el archivo de registro. Su criterio de aceptación es que permita filtrar visualmente por niveles de error (ERROR/FATAL).

La microfase 2.12 desarrolla el motor de exportación de diagnóstico. El objetivo es permitir la extracción del archivo a sistemas externos del usuario de manera offline. Esto requiere implementar un puente estricto utilizando el Storage Access Framework (SAF) nativo. Se invocará Intent.ACTION_CREATE_DOCUMENT, predefiniendo el tipo MIME y sugiriendo el nombre descriptivo fenix-games-diagnostic-[FECHA].txt. El usuario decidirá dónde depositar el archivo sin otorgar permisos heredados peligrosos como WRITE_EXTERNAL_STORAGE.

La microfase 2.13 garantiza la sanitización de datos sensibles. El objetivo es proteger la privacidad durante la extracción de bitácoras. Antes del volcado al SAF, un motor de

expresiones regulares reemplazará nombres reales y metadatos explícitos por alias serializados (por ejemplo, ``). El criterio de aceptación es ético e inflexible.

La microfase 2.14 establece la rotación de logs. El objetivo es evitar que la base de fallos devore el espacio físico del teléfono. Se implementará una rutina que calcule el tamaño límite (ej. 5 MB) del registro acumulado. Si se excede, el sistema aplicará un truncamiento First-In-First-Out, garantizando que el almacenamiento nunca desborde.

La microfase 2.15 incorpora un botón oculto o acceso avanzado. El objetivo es acceder a la pantalla diagnóstica sin perturbar el diseño inmersivo para usuarios casuales. Se añadirá un modificador de gestos combinados (por ejemplo, múltiples toques continuos en la cabecera de la pantalla Home) que desbloquee el enrutamiento interno.

La microfase 2.16 aborda la validación funcional generando un error artificial. El probador insertará un botón temporal que dispare un RuntimeException. El objetivo es comprobar que el manejador global atrapa la excepción fatal, reinicia silenciosamente o finaliza, y asegura el volcado forense.

La microfase 2.17 concluye con la validación de exportación en móvil real. Se ejecutará el error artificial. Luego de reabrir la aplicación offline, el tester utilizará la interfaz oculta para solicitar la exportación. El criterio de aceptación exige que el SAF levante la vista nativa de Android, permita guardar el archivo en la carpeta "Descargas", y que, al abrirlo con un lector de texto, la traza stack trace de la excepción sea completamente íntegra y anonimizada.

6. Fase 3: Primeros juegos offline

La implementación progresiva del catálogo lúdico inicia con los juegos que operan en flujo puramente secuencial y que no requieren lógicas distribuidas complejas, consolidando el patrón de pase-y-juego (pass-and-play).

La microfase 3.1 desarrolla "Yo Nunca". El flujo consta de la iteración simple de sentencias. El ViewModel solicitará al manager una carta categórica, la pantalla Compose la renderizará con una tipografía prominente. Las acciones MVI disponibles son simplemente el avance (Swipe). El modelo utilizado requerirá consultas atómicas SQLite.

La microfase 3.2 integra "Verdad". El flujo se enfoca en cuestionamientos directos a los perfiles pre-cargados. El Estado MVI asociará la carta aleatoria a un UUID particular y la UI notificará al sujeto activo.

La microfase 3.3 construye "Reto". Implica una estructura análoga a la anterior pero introduce la recolección de penalizaciones. Si un jugador evade la sanción física, la UI debe proveer un evento transaccional MVI que reste puntos en Room asincrónicamente.

La microfase 3.4 amalgama "Verdad o Reto combinado". El ViewModel asimila un estado bifurcado. La UI solicita al jugador seleccionar su ruta. La decisión muta el flujo, forzando una subconsulta SQL que filtra por el enumerador de tipo seleccionado (WHERE type='truth' vs WHERE type='dare').

La microfase 3.5 aborda "Mímica / Charadas". En lugar de implementar un giroscopio propenso a lecturas erróneas, la interfaz Compose dependerá de interacciones táctiles directas de validación de los compañeros de equipo para despachar las intenciones lógicas de "acierto" o "fallo", transicionando el estado del timer de forma desacoplada.

La microfase 3.6 implementa el sistema de puntaje básico. El objetivo es mantener una estructura persistente temporal de victoria. Cada modificación se canaliza como un bloque inmutable (ej. UpdateScoreIntent), alterando el valor numérico en la matriz del estado GameSession.

La microfase 3.7 refuerza la lógica para evitar la repetición de cartas. Como se probó en el core, se vincula exhaustivamente la entidad del mazo. La consulta Room garantiza un recuento en el que los IDs leídos quedan excomulgados en tiempo real de las predicciones de selección. La microfase 3.8 diseña la ScoreScreen. Finalizada una ronda paramétrica o al alcanzar un tope, el NavHost redirige a una vista resumida de barras estadísticas elaboradas con Jetpack Compose animado, demostrando quién dominó la sesión sin acceso a métricas globales cloud. La microfase 3.9 inyecta los registros de eventos en la Caja Negra. Se despacharán rastros cada vez que la sesión inicie, rote o culmine una partida de estos subjugos, comprobando el correcto ensamble de los módulos de la Fase 2 en la práctica lúdica. La microfase 3.10 obliga a una validación offline total. Sin ninguna excepción de red, se comprueba empíricamente que la invocación y el barajado SQLite se efectúe en latencia cero y que los JSON originales han nutrido los componentes de Compose a la perfección. La microfase 3.11 es la validación en móvil real. Se debe efectuar una prueba maratónica de usabilidad (Swipe testing) acelerada. El objetivo de aceptación técnica exige que el ciclo constante de lectura asíncrona de base de datos seguido de la recomposición visual en pantalla no derive en bloqueos del hilo principal (Jank) ni exceda el presupuesto térmico de un hardware modesto, probando la estabilidad estática.

7. Fase 4: Modo picante +18

Esta fase requiere la máxima precisión arquitectónica y ética, exigiendo compuertas lógicas infalibles para bloquear el contenido que cruza los límites de intensidad, actuando todo bajo total desconexión y control humano local.

La microfase 4.1 modela el AdultIntensityLevel. Se añadirá una taxonomía escalar como enum class en Kotlin oscilando desde el nivel 0 (Familiar) hasta el 5 (Extremo). Este metadato será anexado a la migración inmutable de las tablas SQLite y al análisis inicial de JSON.

La microfase 4.2 construye el AgeGate. Este componente Compose funge como el portal interceptor de navegación. Cualquier intención de seleccionar mazos restringidos desplegará un Diálogo Modal o un Bottom Sheet robusto y persuasivo obligando a aserciones éticas preliminares.

La microfase 4.3 despliega el ConsentGate. De manera secuencial, la UI inmovilizará la transición de estado requiriendo que cada uno de los avatares matriculados en la sesión confirme y defina activamente su aprobación individual. El modelo MVI denegará el progreso si persiste algún nodo en falso.

La microfase 4.4 define el ConsentState por jugador. En la memoria de la sesión local se amalgaman las declaraciones. La lógica de evaluación matemática determinará el "Límite Global de Sesión" calculando el techo máximo basado en la declaración de la entidad más conservadora del grupo.

La microfase 4.5 crea la selección de intensidad. Se integrará un Slider visual que ajusta los rangos permitidos, amurallados estrictamente por el cálculo del límite del Consent State grupal, para establecer la atmósfera antes de barajar.

La microfase 4.6 decreta la separación total del contenido. Es la regla absoluta donde la restricción del contenido debe descansar íntegramente sobre las funciones del motor DAO de SQLite. Las entidades están separadas por lógica transaccional; se veda la filtración posterior en memoria RAM para mitigar filtraciones visuales accidentales.

La microfase 4.7 implementa los filtros SQL por intensidad. El archivo de base de datos definirá explícitamente consultas como `@Query("SELECT * FROM cards WHERE intensity_level <=`

:sessionCap ORDER BY RANDOM() LIMIT 1").

La microfase 4.8 impone la funcionalidad del Botón Saltar sin penalización. Consiste en la adición inamovible de una vía de escape reactiva en cada pantalla lúdica. La acción transaccional despachará el avance inmediato, sin infligir caídas en el score, garantizando agencia absoluta.

La microfase 4.9 gestiona el retiro del consentimiento. En la barra superior de la navegación Compose siempre debe coexistir un icono de pánico. Su interacción emitirá una reducción jerárquica del umbral tolerado dentro de la sesión activa al instante.

La microfase 4.10 programa la degradación automática a modo familiar. La consecuencia enlazada de la fase anterior exige que, al activarse la retirada, el orquestador MVI de la sesión actualice reactivamente su barrera al nivel 0, forzando a la UI Compose a desestimar la carta activa asincrónicamente y recuperar un contexto seguro purificado.

La microfase 4.11 procesa los packs adultos locales. Esta fase se apoya en los analizadores de `kotlinx.serialization` pero enruta los recursos restringidos hacia los contenedores clasificados correspondientes dentro del almacenamiento pre-poblado.

La microfase 4.12 acopla el registro en la Caja Negra. Se redactarán trazas forenses genéricas informando reducciones de nivel, sin adjuntar nombres u orígenes (ej. "Rebaja de consentimiento ético ejecutada localmente a las 22:40").

La microfase 4.13 es la validación ética y funcional. Se probarán deliberadamente los límites de la arquitectura MVI forzando recomposiciones durante la degradación automática. El criterio de aceptación declara como fracaso catastrófico si una carta +18 permea tras una revocación.

La microfase 4.14 constituye la prueba en móvil real. El tester ejecutará dinámicas extremas offline en grupos falsos simulados, interrumpiendo abruptamente con botones de Salto y Retiro. El hardware debe responder a la mutación de tablas SQL con fluidez, sin cierres por error lógico y demostrando contención ética autónoma.

8. Fase 5: El Impostor pass-and-play

El ingreso a juegos de naturaleza oculta y roles asimétricos, los cuales tradicionalmente dependen de servidores para repartir variables, se resuelve mediante lógica local y manipulación de entropía para funcionar en régimen "pasar el dispositivo".

La microfase 5.1 instauro el modelo ImpostorWordPair. Esta estructura inmutable alojará el conjunto de datos necesario: la variable mayoritaria (Norma) y la variable desviada (Tabú del Impostor), agrupadas temáticamente y listas para la asignación probabilística.

La microfase 5.2 atiende la selección de jugadores. El flujo se asegura de bloquear el comienzo del motor si la longitud del arreglo de participantes matriculados es inferior a tres entidades, preservando el fundamento axiomático del juego de deducción.

La microfase 5.3 desarrolla el algoritmo de asignación de roles. Basado en el volumen de participantes, un iterador con distribución heurística seleccionará aleatoriamente un único UUID para otorgarle la identidad encubierta. El barajado se asegurará con matemáticas estocásticas (Fisher-Yates) para descartar patrones identificables.

La microfase 5.4 orquesta la pantalla privada de revelado. Jetpack Compose presentará una tarjeta abstracta opaca que oculte la visualización hasta que ocurra la interacción física del titular listado, impidiendo el espionaje periférico.

La microfase 5.5 inyecta confirmación antes de mostrar el rol. Se exigirá una barrera transaccional (ej. botón "Soy [Nombre_del_Jugador]") antes de la descryptación visual de la palabra objetivo en memoria.

La microfase 5.6 codifica el ocultamiento del rol tras la visualización. El estado se mutará hacia enmascarado una vez que el sensor táctil sea liberado o expire un cronómetro subyacente. Esto evita la permanencia del secreto durante la rotación física del celular.

La microfase 5.7 coordina la ronda de discusión. Finalizada la asignación asimétrica, un cronómetro central asume el control del flujo y despacha una vista imparcial inmutable exenta de variables sensibles.

La microfase 5.8 gestiona la votación local en un solo dispositivo. El motor presenta cíclicamente a cada usuario activo una lista anónima de acusación en Compose. Las sumatorias operan ocultas dentro del administrador del estado de sesión.

La microfase 5.9 estructura el cálculo del resultado. La lógica evalúa si la entidad impostora fue desenmascarada por pluralidad de votos o si sobrevivió, dictando la victoria de facciones e inyectando las modificaciones atómicas al score.

La microfase 5.10 acopla la Caja Negra. Se emitirán registros asépticos (INFO) documentando el tamaño del grupo, duración total de la fase deductiva y estado terminal, mitigando fugas logarítmicas de la semántica usada.

La microfase 5.11 impone la validación anti-filtración de información. El criterio de aceptación demanda que las intervenciones perjudiciales del sistema operativo (minimizar aplicación, forzar rotación a modo apaisado, modo oscuro) no fueren la recomposición Compose que devele el rol pasivamente.

La microfase 5.12 es la certificación en móvil real. Se transfiere el dispositivo físico repetidas veces emulando un escenario vibrante. Se constatará la ausencia de desbordamientos de buffer o filtración temporal de variables al intercambiar participantes.

9. Fase 6: Tabú, Ruleta y Trivia

Esta fase introduce mecánicas computacionales avanzadas como temporizadores de alta precisión y cálculos matemáticos para interfaces gráficas interactivas operando íntegramente offline.

La microfase 6.1 modela la TabooCard. Se añade un componente de datos extendido que mapea el concepto central junto con la matriz restrictiva lineal de palabras prohibidas derivadas de JSON.

La microfase 6.2 diseña el TabooEngine. Este módulo orquesta la iteración acelerada y maneja las aserciones afirmativas para recolectar puntuación, acoplado al gestor temporal.

La microfase 6.3 requiere la implementación de un TimerEngine fuerte para Tabú. Debido al escrutinio del estrés, el temporizador emitirá eventos al ViewModel y Compose de manera ininterrumpida. Su arquitectura de coroutines debe ser inmune al recambio del ciclo de vida de UI (resistencia a rotaciones).

La microfase 6.4 incorpora las penalizaciones. Interfaz dinámica inferior que emite intenciones reactivas y asíncronas mutando instantáneamente el balance a negativo si el jugador emite las restricciones semánticas.

La microfase 6.5 implementa el modelo físico Ruleta/Botella. Reemplaza herramientas estocásticas por interacción visual.

La microfase 6.6 desarrolla la animación de la ruleta en Compose. En vez de dependencias externas complejas, se invocan funciones nativas trigonométricas de Kotlin y la API interactiva de Animatable<Float>. Se calcularán áreas de impacto circular, desaceleración por inercia asintótica y se dibujarán divisiones espaciales exactas en el Canvas composable.

La microfase 6.7 extrae la selección aleatoria del jugador. Culminado el frenado paramétrico de

la interpolación de la ruleta, el motor deductivo calcula el ángulo de impacto en el círculo e invoca a la entidad apuntada como el receptor del castigo o turno.

La microfase 6.8 elabora la TriviaQuestion. Data class abstracta con el predicado central interrogativo y los arreglos distractores, poblados transparentemente desde el archivo JSON empaquetado.

La microfase 6.9 construye el TriviaEngine. Operará las consultas relacionales asimétricas evaluando en tiempo constante la validez de la entrada humana frente al vector booleano preestablecido.

La microfase 6.10 otorga puntaje por respuesta correcta. Despacho transaccional inmutable que incrementa el saldo del nodo en juego.

La microfase 6.11 añade el puntaje por velocidad. Se integrará una subrutina de decaimiento logarítmico. Si el jugador contesta antes de una fracción de segundo estipulada leyendo el tic continuo del TimerEngine subyacente, el multiplicador se acentúa.

La microfase 6.12 somete los cálculos a la Caja Negra. Especial atención a divisiones por cero en el logaritmo de tiempo o cuellos de botella del Canvas registrados como advertencias (WARNING).

La microfase 6.13 impone validación en móvil real. La ejecución de la física de la ruleta debe sostener los 60 fps empíricos sin agotar las reservas de batería ni congelar la pantalla táctil bajo repetidas presiones, verificando la sanidad del Dispatchers.Main.

10. Fase 7: Editor de packs e import/export

Desacoplamiento total del modelo cerrado, facultando a los usuarios para modificar el ecosistema internamente sin requerir cuentas de plataformas en la nube.

La microfase 7.1 establece la PackEditorScreen. Despliegue de un layout LazyColumn en Compose conteniendo iteradores dinámicos OutlinedTextField que evaden fugas de memoria al generar listas largas.

La microfase 7.2 programa el CRUD de cartas. Operaciones nativas insert/update/delete ejecutadas directamente sobre el ORM SQLite que almacena temporalmente los esquemas en una tabla customizada de usuario.

La microfase 7.3 añade el CRUD de packs. Organización relacional envolvente (uno a muchos) para identificar subconjuntos y agrupar metadatos identificativos.

La microfase 7.4 introduce validación de esquema. Antes de permitir inyecciones SQLite, la heurística Kotlin evaluará cadenas vacías o ataques de longitud masiva (overflow buffer protections) para mantener la estabilidad del layout Compose.

La microfase 7.5 crea la maquinaria de exportar JSON. Extrae los diccionarios SQL definidos por el usuario, los compila mediante la jerarquía de kotlinx.serialization y los devuelve como un objeto binario comprimido de manera asíncrona.

La microfase 7.6 despliega la exportación a formato propietario .party. Transmuta el payload exportado JSON, asigna la extensión custom, y delega todo al Intent.ACTION_CREATE_DOCUMENT. El SAF Android asume el control solicitando al usuario su repositorio final externo sin requerir vulnerar directorios seguros.

La microfase 7.7 programa la mecánica inversa: Importar JSON. Recepción y escrutinio vía Intent.ACTION_OPEN_DOCUMENT de archivos raw.

La microfase 7.8 decodifica archivos importados .party. Extrae la cadena binaria del InputStream proveído por el manejador SAF e inyecta iterativamente los metadatos al motor de Room en procesos por lote (bulk inserts).

La microfase 7.9 establece un férreo manejo de JSON corrupto. Todo proceso parseador está contenido en runCatching. Si `kotlinx.serialization` detona `SerializationException` por llaves nulas, tipología errónea polimórfica o sintaxis maliciosa, el DAO aborta en modo Rollback.

La microfase 7.10 alimenta los logs de Caja Negra. Registra intentos fallidos de importación remarcando vulneraciones sin abortar la ejecución vital del programa (Exception Handling Control).

La microfase 7.11 exige validación con archivo real. Generar un fichero de texto inflado, renombrarlo e intentar el ciclo de exportar-modificar-importar, escrutando si el parser lo procesa síncronamente.

La microfase 7.12 consolida validación en móvil real. Transferir un fichero físico de un dispositivo local a otro (por Bluetooth o utilería de archivos de Android). Desencadenar el intent SAF y verificar que la matriz SQL se actualice dinámicamente y la vista se regenere reactiva.

11. Fase 8: Sala local multicelular

El pináculo arquitectónico: un ecosistema multi-jugador en malla, de latencia cercana a cero, que abjura por completo de servidores y costos operativos perpetuos operando dentro del metal interno.

La microfase 8.1 inyecta los permisos necesarios. Modificación del Manifiesto obligando declaración de INTERNET (para binding a sockets puros locales), y exigiendo sin concesiones las acreditaciones para Android 14 `FOREGROUND_SERVICE` tipificado como `FOREGROUND_SERVICE_TYPE_CONNECTED_DEVICE` (o equivalente válido para servicios P2P/Sync locales) junto al permiso de CAMERA pasivo.

La microfase 8.2 modela el LocalRoom. La entidad conceptual encargada de portar las IPs subyacentes, identificador de sesión alfanumérico y conteo de topología.

La microfase 8.3 despliega el LocalPeer. Modelo que rastrea la identificación de nodos externos acoplados sin recolectar identidades MAC de red puras por seguridad de privacidad, empleando en su lugar un UUID sintético.

La microfase 8.4 incrusta Ktor Server. Configuración del motor asíncrono (Netty o CIO) para actuar de router local escuchando en un puerto abstracto (por ej. 8080) asignado al adaptador red activo (192.168.X.X).

La microfase 8.5 implementa la bóveda de supervivencia: el Foreground Service. El SO Android penaliza operaciones en segundo plano severamente. Un Servicio invocado mediante `startForeground()` con un Notification Builder inamovible encapsulará el motor Ktor. Esto preserva los sockets intactos cuando el anfitrión transiciona a reposo prolongado o multitarea.

La microfase 8.6 instauro el WebSocket local. Se escala el endpoint HTTP básico configurando la extensión plugin de WebSockets Ktor para tramas bidireccionales full-duplex de muy baja latencia.

La microfase 8.7 configura el comportamiento del Host local. El dispositivo principal adopta la responsabilidad de la "verdad única", procesando y emitiendo los estados unificados MVI global hacia el conducto compartido, sin exponerse externamente.

La microfase 8.8 establece el Cliente local. Resto de terminales inicializan `HttpClient` configurado explícitamente para WebSockets, consumiendo el flujo y reflejándolo unidireccionalmente.

La microfase 8.9 desarrolla el descubrimiento óptico. En lugar de protocolos inestables P2P, la pantalla Compose genera sincrónicamente un Código QR basado en arreglos binarios mediante la librería ligera `zxing-core`. La estructura encapsula el JSON: `{"ip": "192.168.x.x",`

"port": 8080, "sessionId": "UUID"}.

La microfase 8.10 incorpora el Escaneo QR. Los clientes operan instancias transparentes CameraX que leen los flujos fotográficos y delegan el procesamiento al analizador offline de ML Kit Barcode Scanning. Las credenciales se disgregan y se desencadena el intento de adhesión al WebSocket instantáneamente.

La microfase 8.11 crea el Lobby local. La vista MVI transicional refleja de inmediato cómo las entidades satélites van consolidando la población del array visual sin retraso de red apreciable. La microfase 8.12 define el Handshake criptográfico mínimo. Negociación protocolar asimétrica garantizando que terminales foráneas desconectadas que escaneen el código intercepten tramas válidas, desechando solicitudes residuales HTTP.

La microfase 8.13 incrusta un Heartbeat pasivo. Para conjurar conexiones "zombie", Ktor instanciará ping-pongs algorítmicos. La omisión continuada del latido por >15 segundos forzará la purga del socket cliente por asfixia del nodo.

La microfase 8.14 gestiona el cierre de sala ordenado. Una subrutina asíncrona finaliza limpiamente los conductos WebSocket, frena la escucha HTTP, colapsa el motor Ktor CIO y solicita stopForeground al SO liberando recursos íntegramente.

La microfase 8.15 empuja los logs de red hacia la Caja Negra. Se despachan trazas de IPs truncadas, errores de enlace TCP, colisiones y resoluciones de host ininteligibles como eventos Forenses a ser inspeccionados vía SAF en caso de inoperancia.

La microfase 8.16 demanda la validación con dos celulares en misma Wi-Fi. Certificación pura P2P doméstica con un router residencial intermedio demostrando cero intervenciones Cloud.

La microfase 8.17 intensifica la validación técnica forzando hotspot. El dispositivo anfitrión aniquila el acceso ISP exterior e invoca un "Punto de Acceso Portátil" Android (Tethering). La IP transita al rango 192.168.43.x. El cliente se conecta y opera lógicamente de forma óptima probando el "Costo Cero Absoluto" operando fuera de cualquier malla comercial.

La microfase 8.18 consolida la validación última de la arquitectura. Minimización continua de host, aperturas de sub-aplicaciones concurrentes, y bloqueos de cristal para verificar la inmutabilidad de Ktor bajo el yugo restrictivo de mitigadores OOM Android.

12. Fase 9: Juegos distribuidos

Consolidado el núcleo P2P asincrónico local, las lógicas monolíticas de deducción matemática abandonan la dependencia física del terminal y se fragmentan.

La microfase 9.1 descompone el "Impostor multicelular". Trasladar el barajado de entropía del hardware solitario hacia un estado distribuido sincronizado, anulando el ritual del pase del terminal.

La microfase 9.2 emite Roles secretos por dispositivo. El motor MVI maestro del servidor host delega, mediante canales WebSocket particulares, los atributos estocásticos segmentados (Unicast), denegando globalmente la carga en la trama JSON generalizada para abolir fisuras y engaños informáticos.

La microfase 9.3 codifica el envío privado de palabra. Reajustar Compose State para reaccionar al payload individual polimórfico deserializado localmente, renderizando el texto sin repercusión en la red principal.

La microfase 9.4 aborda la sincronización de fases. Mutaciones masivas asíncronas dictadas por la base Ktor obligan a la recomposición unísona en todos los clientes (transición alícuota de DISCUSSION -> VOTING).

La microfase 9.5 diseña la Votación privada. La capa de cliente inyecta la elección en el

conductor WebSocket mediante serialización bidireccional asimétrica interceptada en tiempo de ejecución por el ReceiveChannel del Host.

La microfase 9.6 aglutina el Resultado global. Consolidación de matrices asíncronas recibidas antes de decretar la resolución de victoria emitiendo la clausura MVI.

La microfase 9.7 mitiga la intermitencia mediante Reconexión simple. El cliente amnésico (por fallo de WiFi) preserva en DataStore el UUID de sesión; reconecta en caso de Heartbeat truncado, y el host despacha la traza relacional viva para inyectarse de inmediato a la partida sin desbaratar la topología.

La microfase 9.8 resuelve el Manejo de jugador desconectado. Un nodo irrecuperable es expurgado paramétricamente por Ktor para que la matemática sumatoria en votaciones distribuidas no entre en bloqueos perpetuos por votos faltantes.

La microfase 9.9 acopla el registro en Caja Negra. Auditar discrepancias críticas serializadas o caídas abruptas de sockets por latencias exacerbadas, apuntándolos como métricas de red.

La microfase 9.10 estipula la validación mínima distribuida. Operatividad fluida con terminales reducidos blindando que los secretos de la topología asimétrica jamás crucen de la hebra MVI del esclavo B hacia la del esclavo C.

La microfase 9.11 certifica la robustez extrema con tres o más celulares. Escalar iterativamente el estrés probando el colapso asíncrono por concurrencia sobre la infraestructura local y confirmando que el Dispatcher.IO acoplado a Netty/CIO disipa el cuello de botella eficientemente.

13. Estructura final del proyecto

La escalabilidad en Android, ausente de marcos MVC legados y arquitecturas Web frágiles, exige un enrutamiento en árbol puramente regido por Clean Architecture, permitiendo al sistema modular desacoplarse del origen de los datos de su representación visual. La jerarquía oficial de paquetes obligatoria bajo `com.fenixgames.app` será:

- **core:** Infraestructura subyacente agnóstica a los modelos del negocio.
 - **core.engine:** Aloja el cerebro orquestador (SessionManager), el bucle MVI interactivo global que engulle intenciones UI para reducir estados asíncronos en crudo.
 - **core.timer:** Encapsula los cronómetros asíncronos y robustos operados en flujos continuos (StateFlow), proveyendo inmunidad ante el caos de los ciclos de vida de Actividades.
 - **core.random:** Abstención de aleatoriedades predecibles; inyecta Fisher-Yates matemático puro para distribuciones justas de cartas o asignaciones asimétricas de espionaje.
 - **core.validation:** Motores defensivos heurísticos para rechazar estructuras JSON nocivas al ser ingestas por APIs SAF.
 - **core.blackbox:** Motor asíncrono de intercepción logarítmica (IO Writer). Aloja la reescritura forense profunda de capturas globales (UncaughtExceptionHandler) salvaguardando todo rastro sin Internet.
 - **core.network:** Módulo central de interconexión intramuros y resiliencia offline.
 - **core.network.localhost:** Instanciación pesada Ktor (CIO), dependiente vital del ciclo vital superior persistente ForegroundService nativo.
 - **core.network.websocket:** Modelado de tramas complejas, encriptación en vuelo, serialización de carga útil y protocolos Heartbeat Ktor (Ping/Pong).

- core.network.qr: Motor visual sin nube. ZXing renderiza el IP matriz estático en Compose; CameraX y ML Kit Barcode decodifican fotogramas neuronalmente.
 - core.storage: Control de intenciones ACTION_CREATE_DOCUMENT, abstraído desde la dependencia del SAF permitiendo interactividad del explorador Android de usuario, más wrappers ligeros DataStore.
 - core.security: Sanitización (regex masking) de logs exportables para resguardar la identidad (PII) bajo la GDPR.
- data: Bloque portador de conocimiento del sistema persistente.
 - data.local: Definición puramente relacional del ORM offline perpetuo, exento de retrofits web.
 - dao: Objetos de Acceso de Datos amurallados con reglas estrictas de intensidad SQL (e.g. intensity <= 0).
 - entities: Esquemas físicos Room que alojan relaciones foráneas de mazos y componentes.
 - mappers: Transductores lógicos desde DTOs estructurados a entidades operables por Room.
 - data.packs: Parser inicial asíncrono que digiere iterativamente las gigantescas cargas de assets/.
- domain: La médula espinal abstracta de la lógica lúdica; inmutable ante mutaciones UI o cambios DB.
 - domain.models: Clases puras polimórficas (Data y Sealed Classes) desprovistas de marcadores Room o Compose.
 - domain.repositories: Interfaces abstractas de contrato invertido que ocultan la proveniencia SQL de la data al MVI.
 - domain.usecases: Instancias transaccionales unívocas y testables a cargo del delegador de flujo.
- games: Implementación segregada en contenedores estancos independientes.
 - games.charades: Máquina de eventos afirmativos pasiva.
 - games.neverhaveiever: Cíclica lineal apoyada en deducción DAO para evadir lecturas duplicadas.
 - games.truthordare: Parámetros bifurcados que modifican historial inmutable en memoria.
 - games.impostor: Deducción matemática asimétrica pura operando bajo opacidad visual de UI.
 - games.taboo: Matriz dinámica penalizadora asincrónica operada contra latidos milisegundo del TimerEngine.
 - games.roulette: Lógica de animación inercial Animatable<Float>, interpolaciones visuales en base logarítmica Canvas Compose y eventos determinísticos al freno físico.
 - games.trivia: Algoritmos SQL anidados unidos a ponderación en premios por reflejos (latencia decaimiento temporal).
- safety: Entorno de barreras paramétricas del escudo social ético del ecosistema Fénix.
 - safety.consent: Modalidades MVI de intercepción (ConsentGate BottomSheets) y transacciones mutables irrevocables forzando rebaja familiar a nivel núcleo Room.
- ui: Interfaz declarativa agnóstica de hardware.
 - ui.theme: Topología central paramétrica Color/Typo para componentes de Jetpack Compose bajo la doctrina dinámica Material 3.

- ui.components: Bloques de construcción visual atómicos y puros (Botones, Tarjetas re-utilizables) libres de gestión de Estados lógicos.
- ui.navigation: El mapa maestro de NavHost libre de librerías extrañas de migración profunda XML fragmentada.
- ui.screens: Paneles contenedores globales para MVI acoplando estado en composables reactivos superiores mediante variables observables derivadas.
- diagnostics: Interfaz oculta rústica text-base invocable vía paramétrica oculta (Easter Egg) revelando secuencias fallidas y exponiendo botón nativo SAF Android de extracción local cruda.

14. Backlog ejecutable

Este listado priorizado encadena las misiones arquitectónicas operando en ciclos de validación estrictos y sin deudas técnicas arrastradas. Todo trabajo listado elude obligatoriamente ecosistemas Web no embebidos.

ID	Fase	Micro	Tarea Técnica Estricta	Prioridad	Depende de	Archivos Afectados Principalmente	Criterio de Aceptación Crítico
T01	0	0.1	Inicialización del entorno Android puro base	Máxima	-	settings.gradle.kts	KTS Sincroniza. Ejecución CLI build impoluta.
T02	0	0.2	Version Catalogs Unificado libs.versions	Alta	T01	libs.versions.toml, Gradle	Sustitución absoluta de magic strings con variables ref.
T03	0	0.4	Insertión Jetpack Compose + Material 3 dinámico	Alta	T02	build.gradle.kts, Theme.kt	Compilación de nodo HomeScreen abstracto vacío renderiza sin fatalidades.
T04	0	0.7	Integración Asíncrona KSP Room ORM nativo	Máxima	T03	AppDatabase.kt, Config	Anotador de Simbolización inserto y esquema Base Sqlite testable en disco.

ID	Fase	Micro	Tarea Técnica Estricta	Prioridad	Depende de	Archivos Afectados Principalmente	Criterio de Aceptación Crítico
T05	0	0.9	Engine Polimórfico Nativo kotlinx.serialization	Alta	T02	build.gradle .kts, Models	Entiende y extrae esquemas lógicos sin invocar librerías extrañas a Kotlin.
T06	1	1.1	Modelado Domain Models y Entidades Puras	Alta	T05	GameMode .kt, Player.kt	Clases inmutables creadas ignorando arquitectur as ajenas o anotadores visuales UI.
T07	1	1.10	Parser Estático JSON en RAM Hilo IO asíncrono	Máxima	T05	ContentPackManager.kt	Absorbe >1000 nodos estáticos sin congelació n Main Thread (Jankiness)
T08	1	1.11	Pre-populat ion SQL Callback de Room offline	Crítica	T04, T07	AppDatabase.kt Callback	InsertAll() asíncrono no-bloquea nte al abrir App inicial consolidan do data matriz en disco.
T09	1	1.12	Controlador de exclusión Mazo (Set de Repetición)	Media	T04	UsedCardManager.kt, DAOs	Cláusula NOT IN(UUID) filtrada matemática mente sin sobrecarga

ID	Fase	Micro	Tarea Técnica Estricta	Prioridad	Depende de	Archivos Afectados Principalmente	Criterio de Aceptación Crítico
T10	1	1.13	Bucle maestro Orquestador SessionManager MVI	Máxima	T06	SessionManager.kt StateFlow	Consolidar las peticiones y flujos asíncronos y escuchar vistas lúdicas en tiempo real.
T11	2	2.1	File System Writer Asíncrono BlackBoxLogger	Crítica	T01	BlackBoxLogger.kt	Generador e inyector cíclico a un archivo interno de texto estático filesDir.
T12	2	2.6	Capturador Interceptor de Fatal Crash Nativo	Crítica	T11	App.kt	Reemplazo absoluto UncaughtExceptionHandler atrapa Trace, salva asíncrono y aborta proceso limpiamente.
T13	2	2.12	Enrutamiento a Android File System Extractor SAF	Alta	T11	SAF Export Intents	Inyecta y solicita Storage Nativo Android ACTION_CREATE_DOCUMENT escribiendo archivo físico legible sin permisos

ID	Fase	Micro	Tarea Técnica Estricta	Prioridad	Depende de	Archivos Afectados Principalmente	Criterio de Aceptación Crítico
							obsoletos.
T14	2	2.14	Auto-truncado LIFO (Rotación Logs Cíclica)	Media	T11	LocalLogStorage.kt	Destructor paramétrico de colas crónicas si base supera el tope 5MB predeterminado.
T15	3	3.1	MVI ViewModel Engine "Yo Nunca" Compose Pura	Alta	T10	NeverHaveIEverVM.kt	Swiping Gestural interactivo muta índice base StateFlow y absorbe nueva card mitigando crashes visuales.
T16	3	3.4	Motor de bifurcación "Verdad o Reto" SQLite	Media	T04	TruthOrDareVM.kt, DAOs	Elige subconsulta al vuelo WHERE type='truth/ dare' según interacción visual reactiva del usuario.
T17	3	3.8	Módulo Estadístico Analítico de UI ScoreScreen	Baja	T15	ScoreScreen.kt	Mapeo visual reactivo estadístico procesando arreglo List<Player > al finalizar ciclo de mazo.
T18	4	4.2	Puerta	Alta	T03	ConsentGa	UI

ID	Fase	Micro	Tarea Técnica Estricta	Prioridad	Depende de	Archivos Afectados Principalmente	Criterio de Aceptación Crítico
			Defensiva Modal Compose AgeGate			te.kt Sheet UI	Component BottomSheet et interviene todo NavHost exigiendo rúbrica explícita antes de ceder el paso lógico.
T19	4	4.6	Cortafuegos SQL Físico Barrera Intensity Adult	Crítica	T04	CardsDao.kt Queries	Arquitectura amurallada en la consulta SQL subyacente impidiendo la permeabilidad a RAM de intensity=5 sin permiso grupal.
T20	4	4.9	Retracción MVI Downgrade en tiempo real	Alta	T18	SessionManager.kt Action	Recomposición inmediata e instanciación a nivel familiar (cero) tras accionar la interfaz de evasión.
T21	5	5.3	Aleatorización Fisher-Yates P2P Rol Oculto	Media	T01	Randomizer.kt, Impostor	Procesador matemático entropía balanceada e infalible

ID	Fase	Micro	Tarea Técnica Estricta	Prioridad	Depende de	Archivos Afectados Principalmente	Criterio de Aceptación Crítico
							(sin logs de espionaje del resultado).
T22	5	5.4	Pantalla de Sensor Capacitivo Tap-to-view	Media	T21	SecretRoleScreen.kt	Pointer input interactivo difumina secretamente la palabra oculta al soltar el hardware capacitivo (finger-off).
T23	6	6.3	Coroutine TimerEngine Persistente Fuerte	Alta	T03	TimerEngine.kt StateFlow	Generador de flujos tick 1Hz inmutable mitigador de Activity Drops Android rotaciones OS extrema.
T24	6	6.6	Fricción Matemáticas Gráficas Canvas Ruleta	Baja	T03	RouletteView.kt	Uso interpolado de Animatable <Float> sin tirones gráficos (Jank free) a puro polígono Compose.
T25	6	6.11	Decaimiento logarítmico sumatorio Latencia	Baja	T23	TriviaEngine.kt	Concatena respuestas correctas al TimerEngine

ID	Fase	Micro	Tarea Técnica Estricta	Prioridad	Depende de	Archivos Afectados Principalmente	Criterio de Aceptación Crítico
							despachando variabilidad matemática de puntos basada en milisegundos reactivos.
T26	7	7.1	UI Compose Formulario LazyList Pack Creator	Baja	T03	PackEditor Screen.kt	Absorción inputs de OutlinedText Field amparada en Listas perezosas mitigando Overload Memory.
T27	7	7.6	Parser SAF Generador Compresión formato .party	Media	T13	SAF Intents Serialización	Volcado de las entidades personaliza das usuario a binario y extensión propietaria hacia sistema nativo exterior OS.
T28	7	7.9	Rollback Controlador de Excepciones JSON SAF	Alta	T05	P[span_33] (start_span)[span_33](end_span)[span_38](start art_span)[s pan_38](end d_span)ack Manager.kt Flow	Contención segura de (bloque si kotlinx.seri [span_113 (start_spa n)[span_11 3](end_spa n)lization arroja Serializatio

ID	Fase	Micro	Tarea Técnica Estricta	Prioridad	Depende de	Archivos Afectados Principalmente	Criterio de Aceptación Crítico
							nException esquemas polimórficos corruptos.
T29	8	8.1	Modificación de Permisos Manifiesto de Android	Máxima	-	AndroidManifest.xml	Inyección de Internet pasivo, CameraX y servicios imperativos de OS en 1º Plano P2P.
T30	8	8.5	Foreground Service Bóveda Persistencia OS nativa	Crítica	T29	ServerService.kt	El Sistema Operativo mantiene vivo e inexpugnable el Servidor Ktor al fondo, ruteado por Notificación forzosa de persistencia.
T31	8	8.4	Motor Server Asíncrono Ktor Local Embebido CIO	Crítica	T30	HostServer.kt	Instanciación HTTP Listener abstraído en 0.0.0.0:8080 port.
T32	8	8.6	Conector Protocolar WebSocket Endpoints Sync	Crítica	T31	WebSocketRoute.kt	Migración de ruta Ktor a Full-Duplex TCP WS canal de bajísima latencia intralocal.

ID	Fase	Micro	Tarea Técnica Estricta	Prioridad	Depende de	Archivos Afectados Principalmente	Criterio de Aceptación Crítico
T33	8	8.9	Renderizado or Compose ZXing Generador Bitmaps	Media	T03	QrGenerator.kt	Compilación de URI de IP privada JSON a render de Píxeles crudos in situ (sin librerías webs de creación QR).
T34	8	8.10	Neural Escáner Óptico CameraX + ML Kit Offline	Alta	T29	QrScannerScreen.kt	Procesamiento ML On-Device para captación pasiva y extracción de variables IP de conexión inmediata.
T35	8	8.13	Latido Asincrónico Prevención de Fugas (Heartbeat)	Alta	T32	SessionSync.kt Ping-Pong	Generador iterativo de control (Ping Ktor) expulsando zombis que saturan el pipeline de la sala.
T36	9	9.2	Emisor Multicast Asimétrico (Payload Rol Secreto)	Crítica	T32	NetworkEngine.kt	Host orquesta Unicast WebSocket disparando Polimorfismo kotlinx privado a canales

ID	Fase	Micro	Tarea Técnica Estricta	Prioridad	Depende de	Archivos Afectados Principalme nte	Criterio de Aceptación Crítico
							identificado s para abolir robo de estado MVI.
T37	9	9.4	Orquestado r Sincronizac ión Master MVI UI Remota	Alta	T36	SyncMana ger.kt	Flujos remotos controlados por el Host que mutan la pantalla Compose Esclava reactivame nte en avalancha LOBBY->V OTING.
T38	9	9.7	Failsafe Resiliencia Retorno Conexión (Reconnect)	Media	T35	ClientSock et.kt	El nodo reingresa y UUID exige estado inmutable histórico evitando el colapso perenne de partida por intermitenci a de radio.
T39	2	2.15	Inyección Component e Secreto (Easter Egg Logger)	Baja	T11	HomeScre en.kt Nav	7 clics veloces táctiles activan invocación de Diagnostic Screen al probador Android (sin saturar al usuario).
T40	16	16.1	Aserción	Máxima	T01	Hardware	Todas las

ID	Fase	Micro	Tarea Técnica Estricta	Prioridad	Depende de	Archivos Afectados Principalmente	Criterio de Aceptación Crítico
			de Vuelo QA Real Checklists Testing			Físico Fénix	fases atravesadas sin uso de emuladores para Build Final.
<i>(Nota: Este fragmento priorizado destila 40 labores absolutas y nucleares para ensamblar el motor. Las 20 microfases secundarias accesorias restantes engloban labores de ajuste estético, redacción de documentación y tests automatizados de unidad en Kotlin).</i>							

15. Validaciones por fase

Cada fase de código culminará frente a una valla de detención imperativa y exhaustiva para imposibilitar el arrastre subyacente de deuda técnica. Un error no mitigado descalifica la evolución a la fase siguiente.

Validación Fase 0 a Fase 1: El Fundamento Offline

- **Validación Build:** Comando `./gradlew assembleRelease` superado en tiempo razonable. Artifacts compilados que no superen 15MB.
- **Validación Funcional:** Análisis y deglución de assets/ operando asíncrono, transfiriendo 1000+ filas JSON a las capas SQL subyacentes limpiamente.
- **Validación Offline:** Terminal encapsulada. Todos los bloqueos de cortafuegos Android activados; aplicación emana fluida y responsiva a pesar del vacío satelital.
- **Validación Persistencia:** El sistema aniquila la App mediante forzado (Swipe up del Recent Apps Task Manager). Reapertura revive instantáneamente la misma vista por invocación DataStore Room.
- **Validación UI:** Rotaciones físicas agresivas apaisado/retrato validan la impermeabilidad de la variable observable de UI Compose derivada de StateFlow; jamás parpadea a blanco prolongado.

Validación Fase 2: Caja Forense Fénix

- **Validación Funcional:** Pulsador paramétrico inyecta `NullPointerException` explícito y fatídico en el bucle principal (Main Thread).
- **Validación Caja Negra:** Pantalla colapsa abruptamente o transiciona, pero no cuelga infinitamente al SO. El manejador salva traza detallada.
- **Validación UI:** SAF UI Navigator invocado en limpio y adaptativo, obedeciendo la visual Material del explorador nativo Android.
- **Validación Persistencia / Móvil:** Exhumación del `.txt` crudo rotativo (evitando exceder 5MB base). Test de edición externa comprueba que variables e identidades reales sucumbieron bajo el regex de enmascaramiento asimétrico.

Validación Fase 3 a Fase 6: Dinámica e Integridad Gráfica y Visual

- **Validación Funcional:** Entropía y Fisher-Yates iteran masivamente mazos desprovistos de patrones cíclicos solapados en partidas simultáneas probadas a velocidad X10.
- **Validación UI:** Compose Canvas Animatable Ruleta somete a Android Profiler: Cero Jank, Cero Frame-Drops, estricto control de interpolación mantenida a 60 cuadros constantes.
- **Validación Offline:** Cronómetros lúdicos y `TimerEngin[span_158](start_span)[span_158](end_span)` e son acosados por modo doze OS y no resetean ni exponen colapsos de división matemática al reiniciar UI context.

Validación Fase Ética (+18)

- **Validación Funcional:** Intercept paramétrico visual (Bottom Sheet) no se destruye ni omite al golpear la tecla Android "Atrás" `BackHandler`. Inapelabilidad absoluta.
- **Validación Persistencia:** Reducción manual o anulación del acuerdo ético purga reactivamente la intensidad base a cero, expulsando de RAM entidades restringidas.
- **Validación Móvil Real:** Tras purga extrema de Mazo, SQLite devuelve un `EmptyResultSet`. La UI Compose lo absorbe grácilmente escupiendo aviso de mazo agotado familiar sin reventar por índices inválidos (`IndexOutOfBoundsException`).

Validación Fase 8 a Fase 9: Malla Intralocal Descentralizada Ktor

- **Validación Funcional:** Matrices binarias zxing escupen contrastes negros nítidos decodificados sub-300ms por ML Kit de la contraparte.
- **Validación Build:** Manifiesto embebe inamovible la estipulación ServiceInfo Android 14 FOREGROUND_SERVICE_TYPE_CONNECTED_DEVICE esquivando expulsiones Google Play/SO OS.
- **Validación Móvil Real:** Enrutamiento Tethering P2P esclavo a Host sin red exterior pacta Handshakes de apretón inferior a 30ms. MVI opera remoto sin firebase en latencia real-time nula.

16. Validación última en móvil real

El procedimiento de auditoría conclusiva ("Flight QA Checklist") operará bajo las siguientes directivas, asumiendo el rol del usuario final operando al límite. **Pre-condición irrenunciable:** Uso de dos microprocesadores móviles físicos en el extremo opuesto del espectro de poder (hardware top tier vs dispositivo económico obsoleto) con conectividad a Internet totalmente anulada en la red local.

1. **Purga Radical Inicial:** Ejecutar instalación limpia APK. En configuración Android, borrar todos los Datos de Cache/User Data para simular "Lanzamiento Crudo 0" (First Launch).
2. **Ignición Autónoma y Pre-poblado:** Iniciar bajo modo Avión total. Cuantificar y confirmar lectura sub-milisegundo de los diccionarios masivos Json en Room assets/; la pantalla principal de Jetpack Compose debe ascender fluidamente.
3. **Estrés Cíclico Secuencial:** Declarar Identidades en setup ("Player Alpha", "Player Beta"). Ejecutar MVI cíclico en "Yo Nunca". Disparar ráfagas de 50 Skip Buttons visuales. Auditar perfilamiento para evitar Memory Leaks (Fugas de memoria RAM Compose).
4. **Resistencia Ética Invasiva:** Ensayar intrusiones a mazos reservados (+18). Denegar o retirar consentimiento paramétrico en el clímax lúdico vía cabecera TopBar. Validar la metamorfosis transaccional asíncrona del ORM proveyendo un contexto depurado orgánico en la iteración lúdica inminente sin detención visual.
5. **Auditoría UI bajo caos de rotación:** Inyectar estado de cronómetro vivo en pantalla y activar concurrencias multitarea de Android (Split-screen o Picture in Picture PiP). Intercambiar orientación vertical/horizontal desquiciadamente. Comprobar robustez del StateFlow asíncrono y los motores de persistencia ViewModelScope salvaguardando temporizaciones inmutables.
6. **SAF Editor Data Exfiltration:** Abrir CRUD Local. Diseñar Pack Custom. Traspasar al sistema operativo host con export API ACTION_CREATE_DOCUMENT de archivo renombrado binario .party. Abortar App. Borrar Base Datos Room en UI. Importar Fichero vía SAF ACTION_OPEN_DOCUMENT. Polimorfismo Kotlinx Serialization escruta, purifica e infunde el JSON en las tablas de SQLite sin congelamiento de UI principal.
7. **Simulación de Muerte Forense:** Engatillar RuntimeException manual oculto (Easter Egg táctil). App reinicia. Buscar y exhumar la autopsia del archivo SAF. Demostrar anonimato Regex y sanidad de reporte legible (Stacktrace puro) sin fugas GDPR.
8. **Enjambre P2P Descentralizado sin Nube:**
 - Host Android apaga toda WiFi proveedora ISP y activa Zona Portátil Nativa (Hotspot Tethering).
 - Esclavo Android enlaza. El entorno se encuentra ahora fuera de la red comercial global.
 - Host activa Juego Multicelular Impostor. OS otorga inamovilidad

ForegroundService Ktor Server dictaminado por Android 14. Notificación Viva perenne visible.

- Host renderiza Qr Code estático Compose UI. Esclavo invoca Escáner Cámara Lens ML Kit On-Device.
- El descifrado emite la IP 192.168.43.x encapsulada; WebSocket WS transiciona estado a "CONECTADO". Latencia de Handshake instantánea.
- **Muerte por bloqueo táctil (Lock Screen Test):** Host se apaga físicamente en su pantalla (Sleep Mode). Pasan 120 segundos. El Esclavo debe seguir operando, validando envíos y rebotes (Heartbeat Ktor Pings) corroborando invulnerabilidad técnica total sobre los verdugos de asignación RAM nativos de Android y garantizando Coste Cero Cíclico Operativo.

17. Prompts para IA programadora

Bloques lógicos macro-arquitectónicos con restricciones hiper-directivas para delegar y externalizar la generación asíncrona cognitiva, evitando la "alucinación técnica" que intente inyectar nubes (Firebase/Render) contraviniendo el "Costo Cero Absoluto".

Prompt Fase 0 (Iniciación):

Actúa como Arquitecto Senior Android y Tech Lead Kotlin purista. Desarrolla la Fase 0 del sistema "Fénix Games": Andamiaje de Inicialización. Requerimientos y Prohibiciones estrictos: PROHIBIDO integrar llamadas web, Retrofit, Firebase, Supabase, Vercel, Free tiers o nubes. PROHIBIDO emitir código incompleto o "stubs/placeholders" no compilables. Tarea Técnica: Escribe la arquitectura de inicio. Redacta build.gradle.kts (Project y App) basándote exclusivamente en Version Catalogs (libs.versions.toml). Inserta dependencias obligatorias de Jetpack Compose, Material 3 dinámico, procesador de anotaciones Room KSP, DataStore preferencias y abstracciones kotlinx.serialization JSON. Estructura un NavHost robusto Compose delineando enrutamiento de inicio seguro hacia una "HomeScreen" pasiva, inyectando jerarquía ViewModel básica MVI sin mutación lógica profunda. Proporciona el árbol de directorios raíz exacto (Clean Architecture), los comandos de inicialización Git y el primer formato de commit recomendado para asentar los cimientos limpios de compilación "assembleDebug". NO AVANCES HASTA SUPERAR LA VALIDACIÓN DE BUILD.

Prompt Fase 1 (Core Offline):

Actúa como Arquitecto Senior Android de Flujos MVI Unidireccionales. Construye en detalle la Fase 1: Núcleo Lógico Offline Fénix Games. Requerimientos Estrictos: Cero dependencias externas Web o APIs de consulta remota; la aplicación debe subsistir atrapada en un teléfono en Modo Avión total. Tarea Técnica: Diseña modelos puros de memoria sin anotadores UI/Room: GameMode (Sealed Class), Player con Score e Intensity, y GameSession (Data Classes). Implementa en AppDatabase las Entidades y DAOs SQLite paramétricos. Codifica el asincronismo de ContentPackManager fusionado a kotlinx.serialization capaz de parsear en milisegundos un InputStream local estático /assets/cards.json y orquestar el volcado masivo inicial a través de RoomDatabase.Callback prepopulation no bloqueante en el Thread Principal. Elude repetición matemática usando consultas filtradas NOT IN (UUID). Retorna los Dao, Entidades, Mappers y Archivos de carga inicial asíncrona completos, más instrucciones manuales para verificar su fluidez visual sin lags.

Prompt Fase 2 (Caja Negra):

Actúa como Especialista de Confiabilidad de SO Android y Depuración Offline. Elabora íntegramente la Fase 2: Caja Negra y Forense Fénix. Requerimientos Estrictos: Prohíbo

terminantemente Crashlytics, Sentry, Datadog o soluciones analíticas Cloud corporativas. Tarea Técnica: Codifica un interceptor de vida o muerte reescribiendo la jerarquía superior Thread.setDefaultUncaughtExceptionHandler de la Application class nativa para extraer el StackTrace completo crudo, redirigiéndolo asincrónicamente mediante Dispatchers.IO a almacenamiento logarítmico interno rotacional seguro FIFO (<5MB) en filesDir. Conformar el puente nativo Android Storage Access Framework invocando la directiva Intent.ACTION_CREATE_DOCUMENT para desencadenar UI nativo y exportar un fichero de reporte final fenix-games-diagnostic-[FECHA].txt en el File System local desprovisto de permisos heredados obsoletos de manifiesto. Entrega la Interfaz Application, el BlackBoxLogger con Regex de ofuscación de Strings PII GDPR y su intent de volcado.

Prompt Fase 3 (Juegos Base):

Actúa como Compositor Senior UI Jetpack y Arquitecto de Recomposición Asíncrona. Edifica la Fase 3: Juegos Offline Base ("Verdad o Reto", "Yo Nunca"). Requerimientos Estrictos: Abstención de lógica externa. Operaciones resguardadas. Tarea Técnica: Desarrolla StateFlow MVI para control inmutable en ViewModel. Elabora consulta dinámica Room DAO parametrizada en base a selecciones condicionales WHERE type=:type (Bifurcaciones de Jugabilidad). Diseña layouts interactivos Jetpack Compose (Swiping y Tap triggers) atrincherados detrás de variables remember y derivedStateOf esquivando la trampa letal de recomposición infinita del Main Thread (Jank) y caídas RAM. Elabora ScoreScreen estadístico en barras para cierre de sesión y adjunta logs silenciosos (INFO) al vaciarse arreglos para el Logger. Devuelve ViewModel integral, Estados MVI sellados puros, Screens interactivas gestuales e impón checklist offline para certificar usabilidad rotatoria (Orientation Changes) sin amnesia de variables.

Prompt Fase 4 (Modo +18):

Actúa como Ingeniero de UI Reactiva Paramétrica y Seguridad Ética de Software. Materializa la Fase 4: Bóveda de Consentimiento Picante +18. Requerimientos Estrictos: Cero monitoreo corporativo. Lógica paramétrica estricta local. Tarea Técnica: Conceptualiza escala AdultIntensityLevel de Enum (0 a 5). Esculpe ConsentGate Modal Bottom Sheet implacable interceptando capa NavHost superior Compose que detenga lógicamente la sesión MVI requiriendo rúbrica manual UI afirmativa para escalar intensidad del flujo. El Cortafuegos matriz en Dao SQLite forzará cláusula WHERE intensity <= :sessionCap como barrera absoluta infranqueable en memoria. Modifica el SessionManager inyectando botón retráctil de degradación que al oprimirse aplasta paramétricamente el techo a 0 de forma síncrona, eliminando variables y forzando reseteo UI de tarjeta actual reactivo. Redacta ViewModel lógico blindado, DAO mutado inflexible, y Modal Sheet Compose de aserción. Exige validación visual de no fractura por filtraciones.

Prompt Fase 5 (El Impostor):

Actúa como Especialista en Asignación Matemática Pura e Interacciones Clandestinas UI. Desarrolla Fase 5: El Impostor Pass-and-Play Offline. Requerimientos Estrictos: Motor lúdico aislado para hardware táctil rotativo interhumano sin enrutadores foráneos. Tarea Técnica: Abastece Data Classes modelando variable ImpostorWordPair (Mayoritaria/Tabú). Instituye matriz algorítmica Randomizer basada en permutación entrópica uniforme Fisher-Yates aislando asimétricamente 1 entidad UID en GameSession como objetivo espía frente al bloque Civil. Genera pantalla SecretRoleScreen inyectando Modificadores Compose PointerInput asincrónicos (detectTapGestures), obligando retención háptica capacitiva explícita (finger-down) para despintar e invocar texto censurado, devolviendo enmascaramiento reactivo al soltar contacto previendo ojeadas parasitarias al transicionar el celular en mano. Entrega lógica entrópica pura y flujo inmutable View MVI interactivo Compose.

Prompt Fase 6 (Tabú/Ruleta):

Actúa como Gurú Geométrico en Jetpack Compose y Concurrencia Avanzada de Latencia. Implementa Fase 6: Dinámica de Tabú y Fricción Matemática de Ruleta. Requerimientos Estrictos: NO invocar bibliotecas gráficas comerciales UI externas ni llamadas a nubes de cronometraje. Tarea Técnica: Modela TimerEngine persistente en viewModelScope (Dispatchers.Default) que despache emisiones StateFlow sub-milisegundo impermeables al infame exterminio Lifecycle Android por rotación física del OS. Elabora View interactiva RouletteView de Canvas Kotlin aplicando primitivas inerciales logarítmicas de curva paramétrica en Animatable<Float>, renderizando detención estocástica a 60 fps puros sin desgarrar la fluidez visual. Genera lógica deducida de ángulo que compute identidad humana final. Devuelve Motor Temporizador inalterable y vista geométrica visual interpolada.

Prompt Fase 7 (Editor y SAF):

Actúa como Arquitecto Maestro en Local Storage Framework Android. Despliega Fase 7: Creación Modular, Exportación y Extracción Serializada SAF. Requerimientos Estrictos: Evadir herencia de archivos SharedPreferences y APIs Server CMS en desuso. Tarea Técnica: Orquesta PackEditorScreen CRUD usando Layouts Lazy Compose optimizados para grandes arrays UI TextField. Transmuta entidades relacionales en binarios y extensiones mutadas custom .party. Engatilla la Exportación / Importación delegando autonomía e injerencia física local nativa pura del framework Storage Access Framework de Google Intent.ACTION_OPEN_DOCUMENT e Intent.ACTION_CREATE_DOCUMENT. Todo hilo lector InputStream anidado exige contención de parser Polimórfico kotlinx.serialization envuelto en runCatching forzando abortos puros (Rollbacks) silentes si se diagnostica SerializationException esquemática de diccionarios corruptos asimétricos o nulos. Devuelve Coroutines de Extracción SAF y Parser Polimorfo resiliente.

Prompt Fase 8 (Local Room Ktor):

Actúa como Tech Lead Experto en Arquitecturas Socket Peer-To-Peer Locales (Zero Cloud Cost) de Android 14. Ejecuta Fase 8: LAN Ktor WebSocket y Nodos Ópticos. Requerimientos Estrictos: Negativa radical a infraestructuras web, nubes, firebase o APIs P2P corporativas fallidas. Reglas OS Críticas. Tarea Técnica: Inicializa instancia Ktor CIO Host Server vinculada a Hilo OS Service Foreground Service persistente protegido obligatoriamente por flag de manifiesto moderno FOREGROUND_SERVICE_TYPE_CONNECTED_DEVICE y Notificación inamovible (Android 14 Requirements) impidiendo terminación del enrutador P2P de fondo. Activa WS Socket Route TCP. Genera Matriz Bitmaps nativo en Compose zxing-core encapsulando URI IP:PUERTO. Elabora terminal de descifrado Esclava invocando fotograma CameraX analizado en local por ML Kit Barcode neuronal sin requerir permisos red WAN para handshake inicial instantáneo Unicast. Retorna Ktor Init config, Service Lifecycle seguro OS y Optic Decoder UI. NO PROSIGAS SIN APROBAR COMPILACIÓN Y DESTRUCCIÓN DE APP SIN FUGAS DE SERVICIO.

Prompt Fase 9 (Distribución Descentralizada):

Actúa como Orquestador Maestro de Sistemas Distribuidos StateFlow MVI. Implementa Fase 9: Juegos Distributivos en Topología Estrella Local sin Internet de Coste Cero. Requerimientos Estrictos: Destierro conceptual a dependencias de sincronización Rest Web y Vercel en la nube. Tarea Técnica: El Host Server abandona el flujo secuencial y transiciona roles asimétricos deduciendo variables y expidiendo Payloads de Json a canales WebSocket identificados (Unicast), censurando variables ocultas asimétricas para abortar fraude pirata cliente. Codifica interceptor de Latido cíclico (Ktor Heartbeat Ping-Pong) que depure sockets desconectados salvaguardando sesiones activas RAM mediante identificadores temporales UUID recuperables (Reconexiones). Todos los dispositivos clientes fungirán como Views de

pantalla pasivas de la verdad absoluta que emite e inyecta asincrónicamente el Host mediante el flujo ReceiveChannel. Devuelve Lógica de Sync Remota MVI y Payloads Json mitigadores de desincronización por bloqueos de pantalla OS.

18. Riesgos técnicos y mitigación

El destierro autoimpuesto de arquitecturas de la nube (que abstraen la complejidad operativa delegándola a servidores ajenos) exige enfrentar amenazas sistémicas subyacentes originadas en la fragmentación profunda del "metal" de hardware Android y de las variables del ecosistema inalámbrico físico.

Riesgo Operativo Evaluado	Causa o Factor Detonante	Síntoma Desplegado Observable	Prevención / Estrategia de Mitigación Cero-Cloud	Detección Auditable Caja Negra Fénix
Desalineación Gradle KTS	Incoherencia de versiones entre el Compilador de Compose y el núcleo Kotlin.	IDE interrumpe ensamblado. Fracaso total de pipeline CI/CD GitHub.	Implantar matriz rigurosa de Version Catalogs libs.versions.toml dictatorial, suprimiendo strings hardcodeadas.	N/A (Bloqueo analítico de infraestructura en tiempo de compilación).
Migración SQL Destructiva (Room)	Adición de columnas (ej. Intensity) en Room sin plan auto-migratorio o subida de versión.	Colapso IllegalStateException on instantáneo e irreversible al abrir App.	Forzar inyección paramétrica de esquemas ExportSchema JSON e índices AutoMigrations atómicos en bloque @Database.	Exception Handler vomita traza RoomCorruption y aborta.
Integración JSON Contaminado	SAF importa paquete defectuoso (UGC) modificado maliciosamente por comunidad.	Congelamiento o corrupción masiva del ViewModel Parser.	Englobe runCatching en Dispatchers.IO; Invocación restrictiva polimórfica kotlinx mitigando desbordes sin colgar memoria.	Localiza y aísla SerializationException identificando el payload agresor de SAF.
Assets Locales inalcanzables	Empaquetado compilado defectuoso Android (AAB/APK) careciendo del directorio /assets/.	Pantalla vacía perpetua sin cartas jugables de base al instalar.	Rutina de Verificación Hash y conteo post-arranque que lance fallback mode inicial y evite lectura de	Registro INFO detalla: "Asset File Not Found. Initial Popu [span_54](start_span)[span_54](end_span)[span_63](start_span)[spa

Riesgo Operativo Evaluado	Causa o Factor Detonante	Síntoma Desplegado Observable	Prevención / Estrategia de Mitigación Cero-Cloud	Detección Auditable Caja Negra Fénix
			punteros nulos (NullPointerException).	n_63](end_span)la tion aborted."
Desgaste Repetición de Cartas	Carencia de exclusión matemática en motor de memoria RAM y ORM.	El usuario visualiza la tarjeta X repetidas veces destruyendo inmersión lúdica.	Filtros dinámicos SQL atados al ciclo vida MVI invocando UUID NOT IN (:usedCardsSet) en toda consulta limitadora 1 (Randomizer).	Auditoría INFO documenta número de iteraciones o agotamientos prematuros de mazo por UUIDs truncados.
Amnesia de UI por Rotación Android	Sistema Operativo extermina y recrea la jerarquía visual de Activity tras cambio de orientación física (Horizontal/Vertical).	Estados de puntuación lúdica volatilizados; retornos a cero catastróficos.	Sustracción del Estado paramétrico delegándolo íntegramente a bucles StateFlow asíncronos y robustos amparados en el ViewModel inamovible.	Traza subyacente indica Fragmentación MVI o Anomaly Lifecycle resets injustificados.
Aniquilación Asintomática OOM	Android Memory Manager extirpa el proceso de red secundario (Servidor Host) ante demanda RAM frontal.	Los terminales WebSockets clientes pierden sincronía intempestivamente (Malla Caída).	Host encapsulado forzosamente en ForegroundService Android 14+ blindado con FOREGROUND_SERVICE_TYPE_CONNECTED_DEVICE y Notificación inexpugnable.	Anomalía revelada vía Heartbeat Timeout en ReceiveChannel o caídas del hilo Ktor.
Timers UI Inconsistentes / Jank	Compose UI colapsa recomponiéndose frenéticamente ante Emisiones de milisegundos Timer 60fps.	Lag táctil extremo; Animaciones Ruleta o Ticks parpadean o congelan el render (Jank).	Mutación preventiva usando bloques aislados reactivos derivedStateOf en memoria que restrinjan avalanchas inútiles de UI Recomposition	Profiler Logs indican sobrecarga CPU/GPU por "Skipped Frames" excesivos.

Riesgo Operativo Evaluado	Causa o Factor Detonante	Síntoma Desplegado Observable	Prevención / Estrategia de Mitigación Cero-Cloud	Detección Auditable Caja Negra Fénix
			paramétrica.	
Coerción Social +18 / Fallo Ético	Inyección de cartas explícitas sin el umbral de consentimiento unánime activado.	Vulneración inaceptable; quiebre del acuerdo moral social (Agencia nula).	Acoplamiento absoluto inmodificable del ConsentGate Sheet y barrera estructural infranqueable de Query SQL WHERE intensity<=0.	Monitoreo INFO sobre rebajas (Downgrades Éticos) o cancelaciones asíncronas masivas (Sin identidad atada).
Saturación Logarítmica Física	El sistema volca registros forenses indefinidamente y estrangula los gigabytes remanentes del celular.	Fallos de partición IO, advertencia SO por almacenamiento lleno.	Destructor algorítmico de Cola Cíclica (Truncador FIFO LIFO) aniquilando bytes antediluvianos si umbral cruza la muralla de 5MB base estipulado.	Log arroja WARNING de evento: "Log Rotator Activated - Purging oldest bytes".
Violación Privacidad Logs Sensibles	Exportación manual.txt (SAF) contiene Strings crudos de PII (IDs, Nombres Reales, IP Base).	Vulneración regulaciones GDPR operativas y desconfianza tester.	Pre-procesamiento de Sanitización Asimétrica ofuscando (Regex) cada trazo que huella a identidad (``) previo a volcado FileStream local.	N/A (Estrategia puramente de blindaje arquitectónico del pipeline IO).
Fracaso Intent Extracción SAF	Scoped Storage de Android 11+ rechaza peticiones abstractas de apertura de volúmenes /Downloads por faltas de UID.	Ficheros inexportables; sistema secuestrado en partición oculta.	Delegar la recolección física íntegramente a Intent.ACTION_CREATE_DOCUMENT garantizando autonomía y agencia al explorador del File System Nativo sin permisos restrictivos.	Diagnóstico marca SecurityException Permissions Denied SAF IO Failure.
Permisos de	Soberanía Usuario	Escáner aborta;	Try/Catch en	Logger acusa

Riesgo Operativo Evaluado	Causa o Factor Detonante	Síntoma Desplegado Observable	Prevención / Estrategia de Mitigación Cero-Cloud	Detección Auditable Caja Negra Fénix
Cámara Android Nulos	extingue concesión de Manifest Lente CAMERA OS al ejecutar lector Qr ML Kit.	colisión fatal instanciando visor Viewfinder Jetpack Compose.	Compose interceptando SecurityException, derivando elegantemente al cliente a un Modal "Fallback UI" donde introducir IP paramétrica a mano.	CameraX Initialization Permission Refused.
Crash Motor Ktor CIO Enlaced	Instanciación de Binding TCP/HTTP de puerto choca con otra App local bloqueando el 0.0.0.0:8080.	Negativa de lanzamiento sala Host. Caída inmediata módulo red intralocal.	Subrutina iterativa que detecta BindException mutando puerto +1 (8081, 8082) dinámicamente informando visual a render generador QR Compose.	Error documentado: BindExcepti [span_192](start_span)[span_192](end_span) on Socket Already in Use Port 8080.
Hardware de Red IP Cambiante	Enrutador de vivienda DHCP repacta direcciones MAC expulsando el túnel TCP intercelular.	Malla cortada; Handshakes rotos crónicos en la sesión.	Extirpar intervención enrutador comercial instruyendo "Punto de Acceso Portátil" (Hotspot) nativo en el Host garantizando IP férrea intralocal estática 192.168.43.x.	Traza repetitiva pasiva de ConnectException en bucle.
Decodificador QR ML-Kit Inválido	Densidad de píxeles zxing y foco óptico sucio no provee resolución al Neural ML-Kit.	Intentos de unión P2P frustrados; frustración UX sala lobby.	Optimización UI del marco generativo incrementando redundancia error (Error Correction Level) Qr y limpieza visual de ViewFinder Compose sin brillos de pantalla.	Tiempos muertos asintomáticos excesivos (Latency Scan > 10s WARNING).

Riesgo Operativo Evaluado	Causa o Factor Detonante	Síntoma Desplegado Observable	Prevención / Estrategia de Mitigación Cero-Cloud	Detección Auditable Caja Negra Fénix
Desconexión Asíncrona WebSockets	Ruido de radio WiFi desintegra el handshake TCP de latencia viva entre cliente y Ktor Server local.	Instancias lógicas zombificadas arruinan el flujo asimétrico "El Impostor".	Carga persistente Ping-Pong (Heartbeat) de SessionSync. Expurga terminal inerte reactivamente, salvaguarda estado de MVI en DataStore para restitución de reconexión rápida y limpia.	Eventos de red interceptados SocketTimeoutException Drop-Out.
Degradación Térmica Hardware Lento	Compose UI renderiza sobre un terminal Android gama baja (2GB Ram / Procesador MediaTek antiguo) bajo estrés MVI.	Rendimiento catastrófico Jank, recalentamiento extremo batería e input lag táctil.	Elusión de librerías costosas externas; delegar pesada IO a hilos alternativos puros Dispatcher.IO nativos y simplificar opacidad geométrica de la View de Ruleta Animatable<Float>.	CPU Profiler localiza Frames Saltados masivos alertando colapso de Main Thread.
Errores Espectrales Irreproducibles	Variables caen sin dejar rastro de caída (Silent Failures) que aniquilan la lógica MVI State Flow interna.	Comportamientos abstractos erráticos y pantallas lúdicas vacías irresolubles localmente.	Invasión aséptica y omnipresente del BlackBoxLogger INFO en todos los recodos del ViewModel MVI encapsulando subrutinas en bloques silentes runCatching{} de seguridad máxima.	Trazas sutiles que marcan omisiones lógicas en rutinas complejas (Nulls).

19. Criterios de éxito final

La fase terminal que corona la arquitectura de Fénix Games dictaminará el triunfo absoluto o la inviabilidad del ejecutable técnico compilado (fenix-games.apk). La aplicación ostentará validación de "Producción Aprobada" (Gold Code) única y exclusivamente si absorbe incuestionablemente el dogma de la autonomía tecnológica, la resiliencia algorítmica y la

imposición rigurosa de una economía de "Costo Operativo Mensual Igual a Cero".

Primeramente, la infraestructura automatizada de GitHub Actions deberá ensamblar y escupir artefactos binarios estables y nativos, que prescindan de permisos restrictivos invasivos al interactuar y desplegarse en particiones de hardware físico Android, omitiendo la dictadura de los esquemas de aprobación de tiendas comerciales. La autarquía del software no admite excusas: deberá erigir y asimilar sus masas lúdicas estáticas (JSON base) dentro del Hilo IO local y renderizar su orquestación visual en Jetpack Compose, incluso cuando la tarjeta SIM ha sido erradicada y la interfaz inalámbrica es asfixiada por completo por el rigor de un Modo Avión irrevocable.

El eje de éxito primordial y financiero estipula que Fénix Games ha de extinguir todo pasivo operacional (Costo Cero Absoluto). El motor desestimaré, bloqueará y amputará cualquier intento conceptual de conectarse, emitir telemetría, invocar lógicas de autenticación OAuth, requerir inicios de sesión o descargar consultas de backend hacia proveedores onerosos, bases de datos o funciones en la nube como Firebase, Supabase, Vercel, o APIs transaccionales remotas. La facturación de mantenimiento debe permanecer matemáticamente en \$0.00 perpetuamente.

En lo atinente al contenido y su crecimiento, la app consolidará agencia absoluta a su comunidad distribuyendo el poder creativo a través de su Editor UGC interno (Create, Read, Update, Delete). La topología demostrará maestría explotando el Framework nativo Storage Access Framework (SAF), facilitando de manera autónoma, transparente y offline la importación y compresión de mazos generados, y el volcado auditable paramétrico asimétrico de "Cajas Negras Forenses", asegurando salvaguarda de metadatos privados.

El componente de ética y seguridad algorítmica, vital para el uso social humano, debe ratificarse incuestionable. Los muros lógicos DAO AgeGate y ConsentGate encriptarán las variables intensas (Mode +18), y su revocación en vuelo por parte de las entidades someterá de rodillas instantáneamente al flujo MVI recomponiendo sin fallas gráficas y regresando al refugio del estado familiar subyacente. Los saltos paramétricos reaccionarán exentos de latencia visual, otorgando agencia suprema en un toque (Tap).

Finalmente, la demostración de maestría en ecosistemas distribuidos será el sello culminante. Al enrutar la sala asincrónica P2P en el "Host MVI Server Ktor Embebido" sobrepasando las restricciones asesinas de servicios en fondo Android 14+ con su Foreground Service perenne , el proyecto desterrará la dependencia WebWAN para siempre. Los terminales clientes escanearán los Qr estáticos geométricos y transicionarán con escáneres neuronales ML Kit ciegos, negociando "Handshakes" TCP/WebSocket de latencia nula bajo un simple Tethering Hotspot. Si la malla de juego El Impostor sobrevive bajo esta presión sin que los secretos espías se corrompan entre teléfonos y fluyendo a 60 fps, el ecosistema será certificado exitoso, constituyendo un modelo monolítico imperecedero impenetrable por la obsolescencia comercial de infraestructuras cerradas.

Fuentes citadas

1. Access data offline | Firestore - Firebase - Google, <https://firebase.google.com/docs/firestore/manage-data/enable-offline> 2. Adaptive do's and don'ts | Jetpack Compose - Android Developers, <https://developer.android.com/develop/ui/compose/layouts/adaptive/adaptive-dos-and-donts> 3. Building a Scalable UI with Jetpack Compose: Best Practices & Patterns, <https://ranveergour781.medium.com/building-a-scalable-ui-with-jetpack-compose-best-practices-patterns-1dc5dd3e4ce6> 4. Save data in a local database using Room | App data and files -

Android Developers, <https://developer.android.com/training/data-storage/room> 5. `JsonContentPolymorphicSerializer` | `kotlinx.serialization` – Kotlin Programming Language, <https://kotlinlang.org/api/kotlinx.serialization/kotlinx-serialization-json/kotlinx.serialization.json/-json-content-polymorphic-serializer/> 6. `kotlinx.serialization/docs/polymorphism.md` at master · Kotlin/kotlinx.serialization · GitHub, <https://github.com/Kotlin/kotlinx.serialization/blob/master/docs/polymorphism.md> 7. Access documents and other files from shared storage | App data and files | Android Developers, <https://developer.android.com/training/data-storage/shared/documents-files> 8. KtorConnect - Android Ktor Server-Client Applications - GitHub, <https://github.com/emrepbu/KtorConnect> 9. Guide to Foreground Services on Android 14 - Medium, <https://medium.com/@domen.lanisnik/guide-to-foreground-services-on-android-9d0127dc8f9a> 10. Foreground service types are required - Android Developers, <https://developer.android.com/about/versions/14/changes/fgs-types-required> 11. Build a Modern QR Code Scanner App in Android with Jetpack Compose + CameraX + ML Kit - Medium, <https://medium.com/@appdevinsights/build-a-modern-qr-code-scanner-app-in-android-with-jetpack-compose-camerax-ml-kit-068ba40f6fcf> 12. How to Render Text as a QR code in Jetpack Compose - DEV Community, <https://dev.to/devniiaddy/qr-code-with-jetpack-compose-47e> 13. QR Code using ML kit in Android - Medium, <https://medium.com/@myofficework000/qr-code-using-ml-kit-in-android-78564f02fb5c> 14. Follow best practices | Jetpack Compose - Android Developers, <https://developer.android.com/develop/ui/compose/performance/bestpractices> 15. Jetpack Compose UI Elements and Their Usage: Best Practices for Developers - Medium, <https://medium.com/@sharmapraveen91/jetpack-compose-ui-elements-and-their-usage-best-practices-for-developers-9eff664a3c6e> 16. Handle configuration changes | App architecture - Android Developers, <https://developer.android.com/guide/topics/resources/runtime-changes> 17. Load local Json file in Kotlin Multiplatform Library - Stack Overflow, <https://stackoverflow.com/questions/77821949/load-local-json-file-in-kotlin-multiplatform-library> 18. Pre-populating your app database using Room and JSON file | by Satyajeet Mohalkar, <https://medium.com/@satyajeetmohalkar/pre-populating-your-app-database-using-room-using-json-file-abbcc95140cc3> 19. How to use Room with nested JSON - Medium, <https://medium.com/@nemanja.stamenovic/how-to-use-room-with-nested-json-81dec1df1908> 20. Logging to disk reactively on Android | by Karn Saheb - Medium, <https://medium.com/@karnsaheb/logging-to-disk-reactively-on-android-68c4d0ec489> 21. Understand logging | Android Open Source Project, <https://source.android.com/docs/core/tests/debug/understanding-logging> 22. Log | API reference - Android Developers, <https://developer.android.com/reference/kotlin/android/util/Log> 23. How to handle Android exceptions and avoid application crashes - Sentry Blog, <https://blog.sentry.io/how-to-handle-android-exceptions-and-avoid-application-crashes/> 24. Easier Crash Debugging in Android with a Custom In-App Dialog - ProAndroidDev, <https://proandroiddev.com/easier-crash-debugging-in-android-with-a-custom-in-app-dialog-a50a4dbcc9f5> 25. Open files using the Storage Access Framework | App data and files - Android Developers, <https://developer.android.com/guide/topics/providers/document-provider> 26. Bottom sheets | Jetpack Compose - Android Developers, <https://developer.android.com/develop/ui/compose/components/bottom-sheets> 27. Improving the Modal Bottom Sheet API in Jetpack Compose | by Joost Klitsie, <https://proandroiddev.com/improving-the-modal-bottom-sheet-api-in-jetpack-compose-5ca56901ada8> 28. Migrate your Room database | App data and files - Android Developers, <https://developer.android.com/training/data-storage/room/migrating-db-versions> 29. Android

storage use cases and best practices | App data and files,
<https://developer.android.com/training/data-storage/use-cases> 30. How to create RESTful APIs in Kotlin with Ktor, <https://ktor.io/docs/server-create-restful-apis.html> 31. WebSockets in Ktor Server, <https://ktor.io/docs/server-websockets.html> 32. GitHub - landomen/ForegroundServiceSamples: Simple sample app demonstrating how to use Foreground Services on Android 16., <https://github.com/landomen/ForegroundService14Sample> 33. WebSockets in Ktor Client, <https://ktor.io/docs/client-websockets.html> 34. GitHub - alexzhirkevich/custom-qr-generator: Android library for creating QR codes with logo, custom shapes, colors, background image. Powered by ZXing, <https://github.com/alexzhirkevich/custom-qr-generator> 35. Implementing a Barcode/QR-code Scanner with Jetpack Compose and ML Kit - Medium, <https://medium.com/@aashish33lamsal/implementing-a-barcode-qr-code-scanner-with-jetpack-compose-and-ml-kit-87acdeeacbb0> 36. Seamless Data Exchange Between Android Apps Using Ktor - DEV Community, https://dev.to/jc_jaay/seamless-data-exchange-between-android-apps-using-ktor-5hnh 37. Create a WebSocket application in Kotlin with Ktor, <https://ktor.io/docs/server-create-websocket-application.html> 38. Jetpack Compose Offline-First Architectures | by Dobri Kostadinov ..., <https://proandroiddev.com/jetpack-compose-offline-first-architectures-5495ec6ddfa8> 39. Write crash log offline for android app | by Digvijay Wagh - Medium, <https://medium.com/@wagh.digvijay/write-crash-log-offline-for-android-app-bfeb702ece81>